

BAB 1

PENDAHULUAN

1.1 Latar Belakang

1.2 Rumusan Masalah

Rumusan masalah dari tugas akhir ini adalah sebagai berikut:

- Bagaimana cara membangun perangkat lunak permainan *Colored Queens*?
- Bagaimana membangun solusi permainan *Colored Queens* menggunakan teknik *Backtracking* dan *Particle Swarm Optimization* (PSO)?
- Bagaimana membangun solver untuk permainan colored queen yang mengimplementasikan *Backtracking* dan PSO yang dapat diintegrasikan dengan perangkat lunak yang dibangun?
- Bagaimana kinerja dari solver yang dibangun dalam mencari solusi permainan *Colored Queens*?

1.3 Tujuan

Selanjutnya, tujuan dari tugas akhir ini dapat dirumuskan sebagai berikut:

- Membangun perangkat lunak permainan *Colored Queens*.
- Mempelajari cara membangun solusi permainan Colored Queen menggunakan teknik *Backtracking* dan *Particle Swarm Optimization*.
- Membangun solver untuk permainan colored queen yang mengimplementasikan *Backtracking* dan PSO yang dapat diintegrasikan dengan perangkat lunak yang dibangun.
- Melakukan pengujian untuk mengukur kinerja dari solver yang dibangun dalam mencari solusi permainan *Colored Queens*

1.4 Batasan Masalah

Untuk mempermudah pembuatan template ini, tentu ada hal-hal yang harus dibatasi, misalnya saja bahwa template ini bukan berupa style L^AT_EX pada umumnya (dengan alasannya karena belum mampu jika diminta membuat seperti itu)

1.5 Metodologi

Tentunya akan diisi dengan metodologi yang serius sehingga templatanya terkesan lebih serius.

1 Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim
2 eget quam. Quisque libero justo, consectetur a, feugiat vitae, porttitor eu, libero. Suspendisse
3 sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis
4 velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante
5 ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede
6 ac sem eleifend consectetur. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra
7 ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin
8 mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius,
9 eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

10 1.6 Sistematika Pembahasan

11 Rencananya Bab 2 akan berisi petunjuk penggunaan template dan dasar-dasar L^AT_EX. Mungkin
12 bab 3,4,5 dapt diisi oleh ketiga jurusan, misalnya peraturan dasar skripsi atau pedoman penulisan,
13 tentu jika berkenan. Bab 6 akan diisi dengan kesimpulan, bahwa membuat template ini ternyata
14 sungguh menghabiskan banyak waktu.

BAB 2

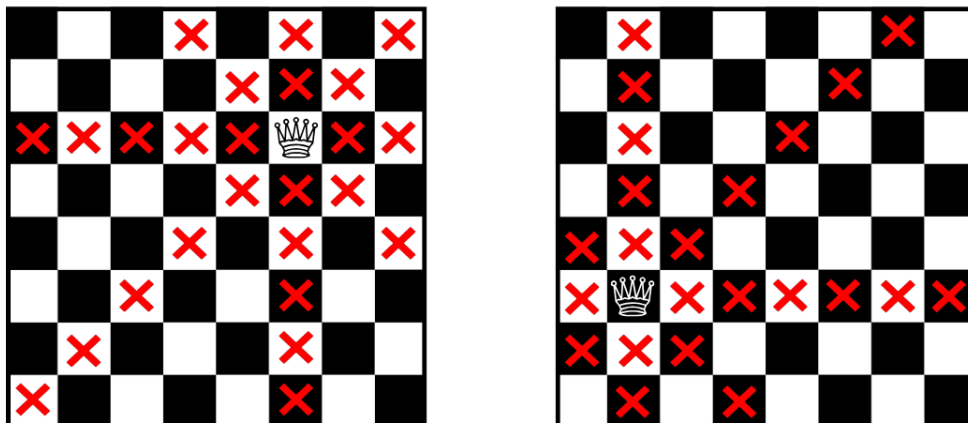
LANDASAN TEORI

2.1 Permasalahan N-Queens

2.1.1 Definisi dan Representasi

Permasalahan N-Queens merupakan salah satu permasalahan klasik dalam bidang ilmu komputer dan matematika diskrit, dengan sejarah panjang yang didokumentasikan secara komprehensif dalam survei yang dilakukan oleh Bell dan Stevens [1]. Permasalahan ini pertama kali diperkenalkan oleh Max Bezzel pada tahun 1848 dalam majalah catur Jerman *Berliner Schachzeitung*. Formulasi awal yang diajukan Bezzel hanya membahas kasus papan berukuran 8×8 , dan baru pada tahun 1850 Franz Nauck mempublikasikan seluruh 92 solusi untuk kasus tersebut sekaligus memperumum permasalahan ke papan berukuran $n \times n$.

Pada formulasi umumnya, diberikan sebuah papan catur berukuran $n \times n$ dan tugasnya adalah menempatkan n buah bidak menteri sedemikian rupa sehingga tidak ada dua menteri yang saling menyerang. Sebagaimana dalam aturan permainan catur, sebuah menteri dapat bergerak secara bebas dalam arah horizontal, vertikal, ataupun diagonal dengan jarak tak terbatas, sehingga setiap penempatan menteri harus mempertimbangkan seluruh arah serangan tersebut seperti yang tertera pada Gambar 2.1.



Gambar 2.1: Contoh aturan penyerangan bidak menteri pada permainan catur. Silang merah menandakan kotak yang terserang oleh bidak menteri.

Dalam praktiknya, papan dapat direpresentasikan menggunakan beberapa cara. Representasi paling intuitif adalah matriks biner berukuran $n \times n$, di mana nilai 1 menandakan keberadaan menteri dan nilai 0 menandakan kotak kosong. Namun, representasi ini tidak efisien dari sudut pandang komputasi karena memungkinkan konfigurasi yang secara trivial tidak valid, seperti dua menteri pada baris yang sama. Oleh sebab itu, representasi yang lebih umum digunakan dalam literatur adalah representasi berbasis permutasi, yaitu sebuah array satu dimensi $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ dengan π_i menyatakan kolom tempat menteri pada baris ke- i ditempatkan [2]. Karena π merupakan permutasi dari himpunan $\{1, 2, \dots, n\}$, representasi ini secara implisit menjamin tidak adanya konflik baris maupun konflik kolom, sehingga ruang pencarian yang perlu dieksplorasi berkurang dari n^n menjadi $n!$ konfigurasi.

Dengan demikian, permasalahan N-Queens pada dasarnya dapat dirumuskan sebagai pencarian permutasi π yang memenuhi kondisi bahwa tidak ada dua menteri yang berbagi diagonal yang sama. Secara formal, π merupakan solusi valid jika dan hanya jika untuk setiap pasangan indeks $i \neq j$ berlaku $|i - j| \neq |\pi_i - \pi_j|$ [2]. Formulasi ringkas ini menjadi titik berangkat bagi pemodelan permasalahan dalam kerangka kerja yang lebih formal, yang akan dibahas pada Bagian 2.3.

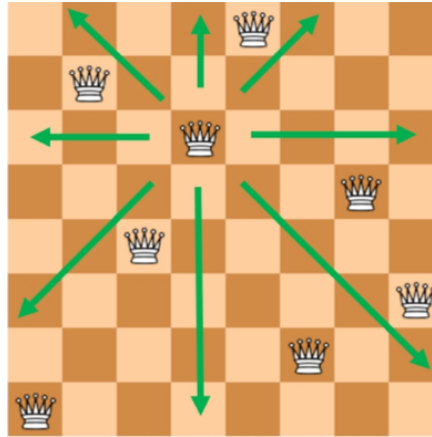
2.1.2 Aturan Konflik Queen

Aturan pergerakan bidak menteri pada permainan catur memunculkan tiga jenis konflik yang harus dihindari secara simultan dalam permasalahan N-Queens. Ketiga jenis konflik tersebut bersesuaian dengan tiga arah pergerakan menteri, yaitu horizontal, vertikal, dan diagonal [1].

Konflik horizontal terjadi apabila terdapat dua menteri yang ditempatkan pada baris yang sama. Mengingat sebuah menteri dapat bergerak bebas sepanjang barisnya, kedua menteri tersebut akan saling menyerang tanpa memandang jarak di antara keduanya. Secara analog, konflik vertikal terjadi apabila dua menteri menempati kolom yang sama. Kedua jenis konflik ini bersifat langsung dan mudah diidentifikasi karena hanya melibatkan kesetaraan indeks baris atau kolom.

Konflik diagonal memerlukan analisis yang sedikit lebih cermat. Sebuah menteri dapat menyerang sepanjang dua jenis diagonal, yaitu diagonal utama yang membentang dari kiri atas ke kanan bawah, serta diagonal anti yang membentang dari kanan atas ke kiri bawah. Untuk dua menteri yang ditempatkan pada posisi (i, π_i) dan (j, π_j) , keduanya berada pada diagonal yang sama apabila selisih baris dan selisih kolomnya memiliki nilai absolut yang setara, yaitu $|i - j| = |\pi_i - \pi_j|$. Karakterisasi aritmatika ini memanfaatkan sifat geometris papan: seluruh kotak pada diagonal utama memiliki nilai $i - \pi_i$ yang konstan, sedangkan seluruh kotak pada diagonal anti memiliki nilai $i + \pi_i$ yang konstan [3].

Berdasarkan ketiga jenis konflik di atas, permasalahan N-Queens dapat dirumuskan secara formal sebagai pencarian penempatan n menteri pada papan $n \times n$ yang memenuhi tiga kendala berikut secara bersamaan: (1) tidak ada dua menteri yang berbagi baris yang sama; (2) tidak ada dua menteri yang berbagi kolom yang sama; dan (3) tidak ada dua menteri yang berbagi diagonal yang sama [3]. Sebagaimana telah dibahas pada Bagian 2.1.1, penggunaan representasi permutasi $\pi = (\pi_1, \pi_2, \dots, \pi_n)$ secara implisit menghilangkan dua kendala pertama, sehingga hanya kendala diagonal yang perlu diperiksa secara eksplisit selama proses pencarian solusi.



Gambar 2.2: Contoh solusi valid permasalahan 8-Queens. Panah hijau menandakan jangkauan serangan salah satu menteri; tidak ada menteri lain yang berada pada baris, kolom, ataupun diagonal yang sama.

Interaksi antarkendala inilah yang membuat permasalahan N-Queens menarik dari sudut pandang komputasi. Pelanggaran terhadap salah satu kendala mengakibatkan konfigurasi menjadi tidak valid, dan kendala-kendala tersebut tidak dapat diperiksa secara terpisah karena keputusan penempatan satu menteri secara langsung memengaruhi domain posisi valid bagi menteri-menteri lainnya. Sifat saling-bergantung antarkendala ini menjadi dasar bagi pemodelan permasalahan menggunakan kerangka kerja *Constraint Satisfaction Problem* yang akan dibahas pada Bagian 2.3.

2.1.3 Kompleksitas dan Relevansi

Dari perspektif komputasi, permasalahan N-Queens menarik karena memiliki struktur yang sederhana namun ruang solusinya sangat besar dan tumbuh secara eksponensial. Apabila tidak ada asumsi tambahan yang digunakan, setiap menteri memiliki n^2 kemungkinan posisi pada papan, menghasilkan ruang pencarian awal sebesar $(n^2)^n$. Penggunaan representasi permutasi sebagaimana dijelaskan pada Bagian 2.1.1 mempersempit ruang ini menjadi $n!$, namun pertumbuhannya tetap superpolynomial. Sebagai gambaran, untuk $n = 14$ terdapat $14! \approx 8,7 \times 10^{10}$ kemungkinan permutasi yang harus dievaluasi, jumlah yang jauh melampaui kemampuan komputasi *brute-force* konvensional pada perangkat keras umum.

Meskipun ruang pencarian tumbuh secara eksponensial, perlu ditekankan bahwa permasalahan N-Queens dalam formulasi dasarnya bukanlah permasalahan yang sulit secara teoretis. Hoffman, Loessi, dan Moore [4] sebagaimana dirangkum dalam survei Bell dan Stevens [1] menunjukkan bahwa terdapat konstruksi aritmatika eksplisit yang dapat menghasilkan satu solusi N-Queens untuk setiap $n \geq 4$ dalam waktu linier $O(n)$ tanpa perlu melakukan pencarian sama sekali. Dengan demikian, persoalan keputusan “apakah solusi N-Queens ada untuk papan $n \times n$ ” dapat dijawab dalam waktu konstan: solusinya selalu ada kecuali untuk $n = 2$ dan $n = 3$.

Karakteristik kompleksitas N-Queens telah menjadi topik diskusi yang panjang dalam komunitas *Artificial Intelligence*, khususnya terkait kelayakannya sebagai *benchmark* algoritma pencarian. Gent, Jefferson, dan Nightingale [5] memberikan analisis komprehensif terhadap persoalan ini dan menunjukkan bahwa permasalahan N-Queens dalam bentuk dasarnya tidak ideal sebagai tolok ukur kesulitan algoritma, mengingat solusinya dapat dikonstruksi secara langsung tanpa

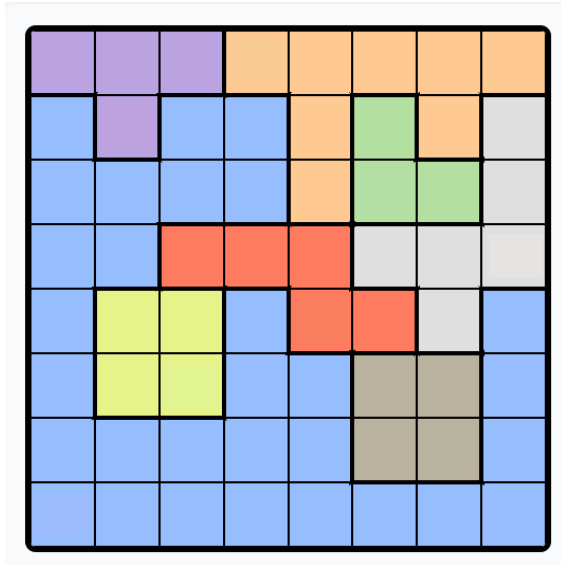
proses pencarian. Sebaliknya, mereka membuktikan bahwa varian *N-Queens Completion*, yaitu permasalahan menyelesaikan konfigurasi parsial yang telah diberikan sebelumnya, tergolong NP-Complete dan #P-Complete. Hasil ini mengindikasikan bahwa varian-varian N-Queens dengan kendala tambahan atau struktur papan ireguler memiliki kompleksitas teoritis yang setara dengan permasalahan-permasalahan sulit klasik dalam ilmu komputer.

Permasalahan *Colored Queens* yang menjadi fokus tugas akhir ini memiliki struktur kendala yang jauh lebih kompleks dibandingkan N-Queens klasik, termasuk pembagian sektor warna dan kendala *adjacency* yang tidak dapat diselesaikan dengan konstruksi aritmatika sederhana. Dengan kata lain, meskipun teknik konstruktif tersedia untuk N-Queens dasar, teknik tersebut tidak dapat diperumum ke varian-varian dengan kendala ireguler. Hal inilah yang menjadikan pendekatan algoritmik berbasis pencarian, baik yang bersifat sistematis maupun *metaheuristic*, tetap relevan dan menjadi fokus pembahasan pada bab-bab selanjutnya.

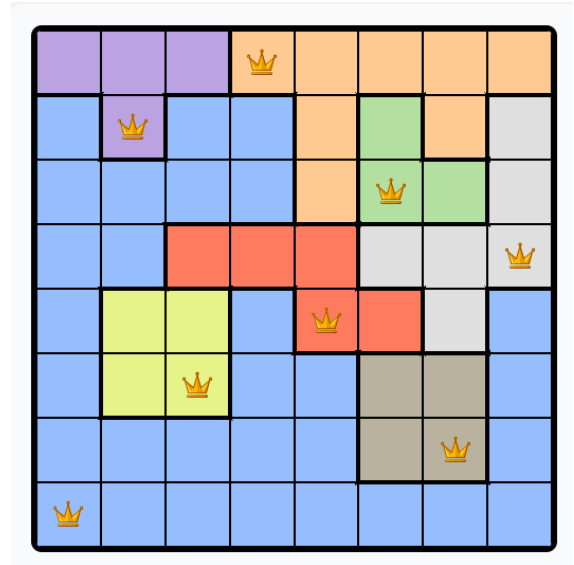
2.2 Colored Queens Problem

2.2.1 Definisi dan Aturan Permainan

Permasalahan *Colored Queens* merupakan sebuah *puzzle* logika berbasis papan yang dipopulerkan melalui permainan *Queens* pada platform LinkedIn [6]. Secara struktural, permasalahan ini memiliki kesamaan yang kuat dengan jenis *puzzle* penempatan objek yang dikenal sebagai *Star Battle*, yang pertama kali dirancang oleh Hans Eendebak untuk *World Puzzle Championship* tahun 2003 [7]. Dalam varian yang dibahas pada tugas akhir ini, permainan menggunakan papan berukuran $n \times n$ yang dibagi menjadi n buah sektor berwarna (*colored regions*) dengan bentuk yang umumnya tidak beraturan (*irregular*). Setiap sektor warna membentuk satu wilayah yang terhubung (*connected region*) pada papan, artinya seluruh sel dalam satu sektor dapat dijangkau dari sel lainnya melalui tetangga horizontal atau vertikal tanpa melewati sel dari sektor lain. Contoh papan permainan beserta solusinya diperlihatkan pada Gambar 2.3.



(a) Papan awal



(b) Papan yang telah diselesaikan

Gambar 2.3: Contoh papan permainan *Colored Queens* berukuran 8×8 dengan 8 sektor warna. Setiap sektor membentuk wilayah terhubung, dan solusi menempatkan tepat satu menteri per baris, kolom, dan sektor warna tanpa ada dua menteri yang bertetangga.

Tujuan permainan adalah menempatkan tepat n buah bidak menteri pada papan sedemikian rupa sehingga seluruh aturan berikut terpenuhi secara bersamaan:

1. Setiap baris memuat tepat satu menteri.
2. Setiap kolom memuat tepat satu menteri.
3. Setiap sektor warna memuat tepat satu menteri.
4. Tidak ada dua menteri yang menempati sel-sel yang bertetangga (*adjacent*), termasuk tetangga diagonal. Secara formal, dua sel (r_1, c_1) dan (r_2, c_2) dikatakan bertetangga apabila $|r_1 - r_2| \leq 1$ dan $|c_1 - c_2| \leq 1$.

Perlu dicatat bahwa, berbeda dengan permasalahan N-Queens klasik yang dibahas pada Bagian 2.1, bidak menteri dalam *Colored Queens* **tidak** menyerang sepanjang diagonal. Serangan hanya berlaku dalam arah horizontal dan vertikal, sehingga kendala diagonal digantikan oleh kendala *adjacency* yang bersifat lokal (hanya memengaruhi delapan sel di sekeliling menteri). Perbedaan ini mengubah karakteristik ruang pencarian secara fundamental, karena kendala *adjacency* tidak dapat direduksi menjadi perbandingan aritmatika sederhana sebagaimana kendala diagonal pada N-Queens.

Selain itu, keberadaan sektor warna yang berbentuk ireguler menambah dimensi kendala yang tidak dimiliki oleh N-Queens. Pada N-Queens, satu-satunya entitas pembatas adalah baris, kolom, dan diagonal yang semuanya memiliki struktur geometris teratur. Pada *Colored Queens*, sektor warna memiliki bentuk yang bervariasi antarpapan dan tidak mengikuti pola geometris yang dapat diprediksi, sehingga setiap instansi permainan memerlukan analisis kendala yang berbeda.

2.2.2 Kompleksitas dan Struktur Masalah

Permasalahan *Colored Queens* memiliki struktur kendala berlapis (*multi-layered constraints*) yang membedakannya secara fundamental dari N-Queens klasik. Sebagaimana diformulasikan oleh

Mata Ali dan Mencia [8], permainan ini merupakan varian dari N-Queens yang melonggarkan kendala diagonal namun menambahkan kendala sektor warna dan *adjacency*. Kombinasi kendala-kendala tersebut menghasilkan interaksi yang lebih kompleks dibandingkan N-Queens, karena setiap keputusan penempatan menteri secara simultan memengaruhi tiga domain pembatas yang berbeda: baris/kolom, sektor warna, dan lingkungan tetangga.

Lapisan pertama adalah kendala baris dan kolom, yang secara struktural identik dengan N-Queens dan membatasi setiap baris serta kolom untuk memuat tepat satu menteri. Lapisan kedua adalah kendala sektor warna, yang mengharuskan setiap sektor memiliki tepat satu menteri. Berbeda dengan kendala baris dan kolom yang bersifat reguler dan simetris, sektor warna memiliki bentuk ireguler yang bervariasi antarpapan. Implikasinya, ukuran domain untuk setiap sektor (jumlah sel yang tersedia) tidak seragam, dan beberapa sektor mungkin memiliki domain yang jauh lebih kecil dibandingkan sektor lainnya. Hal ini menciptakan ketidakseimbangan dalam proses pencarian: penempatan menteri pada sektor berukuran kecil memiliki dampak propagasi yang berbeda dibandingkan penempatan pada sektor berukuran besar.

Lapisan ketiga adalah kendala *adjacency*, yang melarang dua menteri menempati sel-sel yang bertetangga. Kendala ini bersifat lokal dan simetris, namun efeknya bersifat global karena penempatan satu menteri secara langsung mengeliminasi hingga delapan sel tambahan dari domain menteri-menteri lainnya. Pada papan berukuran kecil ($n \leq 8$), eliminasi ini sangat signifikan secara proporsional terhadap total sel yang tersedia.

Interaksi antara ketiga lapisan kendala tersebut menciptakan beberapa tantangan komputasi. Pertama, tidak ada representasi permutasi sederhana sebagaimana pada N-Queens, karena variabel terkait erat dengan sektor warna yang bentuknya tidak teratur. Kedua, kendala *adjacency* tidak dapat diekspresikan sebagai ketidaksamaan aritmatika global (seperti $|i - j| \neq |\pi_i - \pi_j|$ pada N-Queens), melainkan harus diperiksa secara lokal untuk setiap pasangan sel. Ketiga, bentuk ireguler sektor warna menyebabkan *constraint graph* yang dihasilkan tidak memiliki struktur simetris yang dapat dieksploitasi oleh teknik konstruktif.

Dari sudut pandang kompleksitas, meskipun belum terdapat pembuktian formal bahwa *Colored Queens* termasuk dalam kelas NP-Complete, struktur permasalahannya memiliki kemiripan dengan *N-Queens Completion* yang telah terbukti NP-Complete [5]. Keduanya melibatkan kendala tambahan yang merusak keteraturan geometris N-Queens dasar, sehingga teknik konstruktif tidak dapat diterapkan. Kompleksitas inilah yang menjadikan permasalahan ini menarik untuk diselesaikan secara algoritmik menggunakan pendekatan berbasis pencarian.

2.2.3 Posisi dan Relevansi Penelitian

Permasalahan *Colored Queens* merupakan topik yang relatif baru dalam literatur akademis. Meskipun permainan ini telah menarik perhatian luas sejak diperkenalkan oleh LinkedIn sebagai *puzzle* harian [6], kajian ilmiah formal terhadap permasalahan ini masih sangat terbatas. Sejauh tinjauan pustaka yang dilakukan, satu-satunya publikasi akademis yang secara langsung memformulasikan permainan LinkedIn Queens adalah karya Mata Ali dan Mencia [8], yang mengusulkan formulasi *Quadratic Unconstrained Binary Optimization* (QUBO) untuk menyelesaikan generalisasi permainan tersebut pada perangkat keras kuantum. Pendekatan-pendekatan lain yang tersedia di komunitas pengembang umumnya bersifat informal dan tidak dipublikasikan melalui kanal akademis, seperti

implementasi *backtracking* sederhana, formulasi *integer programming*, serta penggunaan *SAT/SMT solver*.

Celah penelitian (*research gap*) yang teridentifikasi dari tinjauan ini adalah sebagai berikut. Pertama, belum terdapat kajian yang memodelkan *Colored Queens* secara eksplisit sebagai *Constraint Satisfaction Problem* (CSP) dan menyelesaikannya menggunakan teknik-teknik klasik CSP seperti *backtracking* dengan *forward checking* dan *arc consistency*. Kedua, belum ditemukan penelitian yang membandingkan pendekatan sistematis (*complete*) dengan pendekatan *metaheuristic* untuk permasalahan ini. Ketiga, analisis empiris mengenai pengaruh ukuran papan terhadap performa algoritma penyelesaian juga belum dilakukan secara formal.

Tugas akhir ini bertujuan untuk mengisi celah tersebut dengan memberikan dua kontribusi utama. Kontribusi pertama adalah pemodelan *Colored Queens* sebagai CSP dan penyelesaiannya menggunakan *backtracking* yang diperkuat dengan *forward checking* dan *Arc Consistency* (AC-3). Kontribusi kedua adalah penerapan *Particle Swarm Optimization* (PSO) yang diadaptasi untuk ruang pencarian diskrit sebagai pendekatan alternatif. Perbandingan antara kedua pendekatan tersebut diharapkan memberikan wawasan mengenai *trade-off* antara kelengkapan solusi dan efisiensi waktu, serta perilaku masing-masing algoritma pada papan berukuran kecil hingga besar.

2.3 Constraint Satisfaction Problem

2.3.1 Definisi dan Komponen CSP

Constraint Satisfaction Problem (CSP) merupakan kerangka kerja formal untuk merepresentasikan dan menyelesaikan permasalahan yang melibatkan sekumpulan variabel dengan batasan-batasan yang harus dipenuhi secara bersamaan. Secara formal, sebuah CSP didefinisikan sebagai tripel $\mathcal{P} = \langle X, D, C \rangle$ [3] di mana:

- $X = \{x_1, x_2, \dots, x_n\}$ adalah himpunan variabel yang nilainya harus ditentukan.
- $D = \{D_1, D_2, \dots, D_n\}$ adalah himpunan domain, di mana D_i menyatakan himpunan nilai yang dapat diambil oleh variabel x_i .
- $C = \{C_1, C_2, \dots, C_m\}$ adalah himpunan kendala, di mana setiap kendala C_j mendefinisikan kombinasi nilai yang diperbolehkan untuk suatu subhimpunan variabel tertentu.

Sebuah *assignment* adalah pemetaan dari satu atau lebih variabel ke nilai-nilai dalam domainnya. *Assignment* disebut *consistent* apabila tidak melanggar kendala manapun, dan disebut *complete* apabila seluruh variabel telah diberi nilai. Solusi dari sebuah CSP adalah *complete assignment* yang bersifat *consistent*, yaitu seluruh variabel telah diberi nilai dan seluruh kendala terpenuhi [3].

Kajian formal mengenai CSP berawal dari karya Montanari pada tahun 1974 yang memperkenalkan kerangka jaringan kendala (*constraint network*), dan dilanjutkan oleh Mackworth pada tahun 1977 yang mengembangkan algoritma *arc consistency* sebagai teknik dasar untuk mereduksi ruang pencarian [9]. Sejak saat itu, CSP telah menjadi paradigma fundamental dalam *Artificial Intelligence* dan digunakan secara luas untuk memodelkan berbagai permasalahan seperti penjadwalan, pewarnaan graf, dan penyelesaian *puzzle* [3].

2.3.2 Teknik Representasi

Sebuah CSP dapat direpresentasikan secara visual menggunakan *constraint graph*, yaitu sebuah graf di mana setiap simpul merepresentasikan variabel dan setiap sisi menghubungkan dua variabel yang terlibat dalam suatu kendala bersama [3]. Representasi ini berguna untuk menganalisis struktur ketergantungan antarvariabel serta menentukan strategi penyelesaian yang tepat.

Kendala dalam CSP dapat diklasifikasikan berdasarkan jumlah variabel yang terlibat. Kendala *unary* melibatkan satu variabel dan membatasi domain variabel tersebut secara langsung, misalnya $x_1 \neq 3$. Kendala *binary* melibatkan dua variabel dan membatasi kombinasi nilai yang diperbolehkan di antara keduanya, misalnya $x_1 \neq x_2$. Dalam praktiknya, sebagian besar CSP dapat ditransformasikan ke dalam bentuk yang hanya mengandung kendala *binary* tanpa kehilangan informasi [3], dan banyak algoritma penyelesaian CSP dirancang secara khusus untuk menangani kendala jenis ini.

Struktur *constraint graph* memiliki implikasi langsung terhadap kompleksitas penyelesaian. Apabila graf memiliki struktur pohon (*tree-structured*), CSP dapat diselesaikan dalam waktu polinomial. Sebaliknya, graf yang padat (*dense*) dengan banyak sisi mengindikasikan interaksi kendala yang tinggi dan umumnya memerlukan teknik penyelesaian yang lebih canggih [3].

2.3.3 Pemodelan Colored Queens sebagai CSP

Permasalahan *Colored Queens* dapat dimodelkan secara natural sebagai CSP dengan mendefinisikan komponen-komponen $\langle X, D, C \rangle$ sebagai berikut.

Himpunan variabel $X = \{x_1, x_2, \dots, x_n\}$ merepresentasikan n buah sektor warna pada papan, di mana setiap variabel x_i bersesuaian dengan sektor warna ke- i . Nilai yang diberikan pada x_i menyatakan posisi sel tempat menteri ditempatkan dalam sektor tersebut.

Domain D_i untuk setiap variabel x_i adalah himpunan seluruh sel yang termasuk dalam sektor warna ke- i . Secara formal, apabila sektor warna ke- i terdiri dari sel-sel $\{(r_1, c_1), (r_2, c_2), \dots, (r_k, c_k)\}$, maka $D_i = \{(r_1, c_1), (r_2, c_2), \dots, (r_k, c_k)\}$. Perlu dicatat bahwa ukuran domain bervariasi antarsektor karena bentuk sektor yang ireguler, sehingga $|D_i| \neq |D_j|$ pada umumnya.

Himpunan kendala C terdiri dari empat jenis kendala yang bersesuaian dengan aturan permainan:

1. **Kendala baris:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, apabila $x_i = (r_i, c_i)$ dan $x_j = (r_j, c_j)$, maka $r_i \neq r_j$.
2. **Kendala kolom:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, berlaku $c_i \neq c_j$.
3. **Kendala sektor warna:** dipenuhi secara implisit oleh definisi variabel, karena setiap variabel hanya dapat mengambil nilai dari sektornya sendiri.
4. **Kendala adjacency:** untuk setiap pasangan variabel x_i, x_j dengan $i \neq j$, apabila $x_i = (r_i, c_i)$ dan $x_j = (r_j, c_j)$, maka $|r_i - r_j| > 1$ atau $|c_i - c_j| > 1$.

Seluruh kendala di atas bersifat *binary*, yaitu melibatkan tepat dua variabel, sehingga permasalahan *Colored Queens* menghasilkan *constraint graph* yang bersifat *complete*—setiap pasangan variabel terhubung oleh setidaknya satu kendala. Karakteristik ini mengindikasikan tingkat interaksi kendala yang tinggi dan memperkuat argumen bahwa teknik propagasi kendala diperlukan untuk mereduksi ruang pencarian secara efektif sebelum atau selama proses pencarian solusi. Teknik-teknik penyelesaian tersebut akan dibahas pada bagian berikutnya.

2.4 Teknik Penyelesaian CSP

Bagian ini membahas teknik-teknik yang digunakan untuk menyelesaikan CSP secara umum, yang kemudian akan diterapkan pada permasalahan *Colored Queens*. Teknik-teknik tersebut dikelompokkan menjadi dua kategori besar: algoritma sistematis yang menjamin kelengkapan pencarian, dan pendekatan *metaheuristic* yang bersifat probabilistik namun mampu menjelajahi ruang solusi yang sangat besar secara efisien.

2.4.1 Algoritma Sistematis

Algoritma sistematis untuk CSP adalah algoritma yang menjamin ditemukannya solusi apabila solusi tersebut ada, atau membuktikan ketiadaan solusi apabila tidak ada satu pun penugasan yang memenuhi seluruh kendala. Jaminan kelengkapan (*completeness*) ini membedakan algoritma sistematis dari pendekatan *metaheuristic* yang akan dibahas pada Bagian 2.4.2. Dalam konteks CSP, keluarga algoritma sistematis yang paling banyak digunakan dibangun di atas fondasi *backtracking search*, yang kemudian diperkuat oleh teknik-teknik propagasi kendala seperti *forward checking* dan *arc consistency* [3].

Backtracking Search

Backtracking search adalah algoritma pencarian berbasis *depth-first search* (DFS) yang membangun solusi CSP secara inkremental, satu variabel dalam satu langkah [3]. Pada setiap langkah, algoritma memilih sebuah variabel yang belum diberi nilai, lalu mencoba setiap nilai dalam domain variabel tersebut secara berurutan. Apabila sebuah nilai tidak melanggar kendala apapun terhadap variabel-variabel yang telah ditetapkan sebelumnya, nilai tersebut ditetapkan dan algoritma melanjutkan ke variabel berikutnya secara rekursif. Apabila tidak ada nilai yang konsisten ditemukan untuk variabel saat ini, algoritma mundur (*backtrack*) ke variabel sebelumnya dan mencoba nilai lain dari domain variabel tersebut.

Pseudocode dasar *backtracking search* diperlihatkan pada Listing 1.

Algorithm 1 Backtracking Search untuk CSP

```

1: function BACKTRACK(assignment, csp)
2:   if assignment lengkap then
3:     return assignment
4:   end if
5:   var ← SELECTUNASSIGNEDVARIABLE(csp)
6:   for setiap value dalam ORDERDOMAINVALUES(var, assignment, csp) do
7:     if value konsisten dengan assignment then
8:       Tambahkan {var = value} ke assignment
9:       result ← BACKTRACK(assignment, csp)
10:      if result ≠ failure then
11:        return result
12:      end if
13:      Hapus {var = value} dari assignment
14:    end if
15:  end for
16:  return failure
17: end function

```

Keunggulan utama *backtracking search* dibandingkan pencarian *brute-force* naif adalah pemangkasan ruang pencarian (*pruning*) yang terjadi secara otomatis: setiap kali penugasan parsial terdeteksi melanggar kendala, seluruh cabang pencarian di bawahnya dieliminasi tanpa perlu dieksplorasi. Secara formal, kompleksitas waktu terburuk dari *backtracking* pada CSP dengan n variabel dan ukuran domain maksimum d adalah $O(d^n)$ [10]. Meskipun tetap eksponensial, angka ini jauh lebih kecil dibandingkan pencarian lengkap yang mengevaluasi seluruh d^n kombinasi tanpa pemangkasan.

Performa *backtracking search* sangat dipengaruhi oleh dua keputusan heuristik: urutan pemilihan variabel dan urutan pemilihan nilai dalam domain. Heuristik *Minimum Remaining Values* (MRV) memilih variabel yang memiliki jumlah nilai valid terkecil dalam domainnya, berdasarkan intuisi bahwa variabel yang paling terbatas cenderung menyebabkan kegagalan lebih awal sehingga pemangkasan terjadi di level pohon pencarian yang lebih tinggi [3]. Di sisi lain, heuristik *Least Constraining Value* (LCV) memilih nilai yang paling sedikit membatasi domain variabel-variabel lainnya, dengan tujuan memaksimalkan fleksibilitas untuk langkah-langkah selanjutnya.

Meskipun heuristik-heuristik tersebut dapat mengurangi jumlah *backtrack* secara signifikan, *backtracking* murni masih rentan terhadap fenomena *thrashing*, yaitu pengulangan kegagalan yang sama berulang kali akibat konflik yang tidak terdeteksi secara dini [10]. Sebagai contoh, apabila dua variabel yang belum ditugaskan memiliki domain yang saling berkonflik, *backtracking* murni baru akan mendeteksi konflik tersebut setelah salah satu dari keduanya diproses, yang mungkin baru terjadi setelah banyak langkah rekursi. Kelemahan inilah yang mendorong pengembangan teknik propagasi kendala yang diintegrasikan ke dalam proses *backtracking*, dimulai dari *forward checking* pada bagian berikutnya.

Forward Checking

Forward checking adalah teknik propagasi kendala yang diintegrasikan ke dalam *backtracking search* untuk mendeteksi kegagalan lebih awal [11]. Ide dasarnya adalah: setiap kali sebuah variabel x_i

diberi nilai v , algoritma segera memeriksa seluruh variabel yang belum diberi nilai dan terhubung dengan x_i melalui suatu kendala, lalu menghapus dari domain variabel-variabel tersebut semua nilai yang tidak konsisten dengan penugasan $x_i = v$. Proses ini dilakukan secara langsung pada saat penugasan, bukan menunggu hingga variabel-variabel tersebut giliran diproses.

Pseudocode *forward checking* diperlihatkan pada Listing 2.

Algorithm 2 Forward Checking

```

1: function FORWARDCHECK( $var, value, assignment, csp$ )
2:   for setiap variabel  $Y$  yang belum ditugaskan dan memiliki kendala dengan  $var$  do
3:     for setiap  $v \in D_Y$  do
4:       if  $v$  tidak konsisten dengan  $value$  berdasarkan kendala  $(var, Y)$  then
5:         Hapus  $v$  dari  $D_Y$ 
6:       end if
7:     end for
8:     if  $D_Y$  kosong then
9:       return failure
10:    end if
11:  end for
12:  return success
13: end function

```

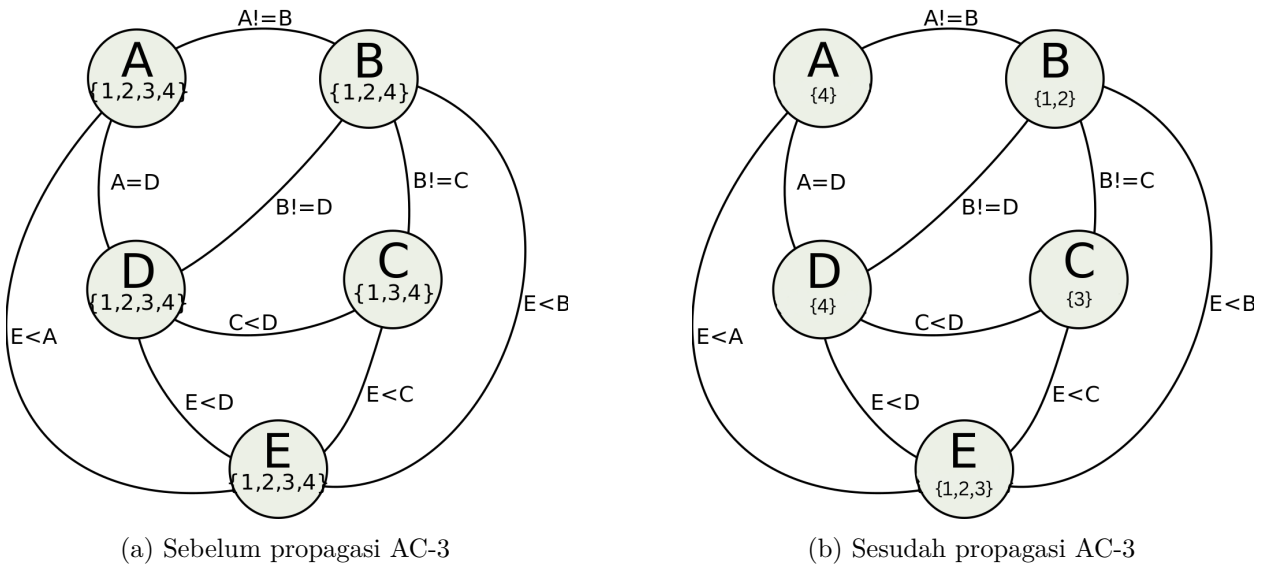
Keuntungan langsung dari *forward checking* adalah kemampuannya untuk mendeteksi *domain wipeout* — yaitu kondisi di mana domain suatu variabel yang belum ditugaskan menjadi kosong — jauh sebelum variabel tersebut giliran diproses oleh rekursi. Apabila *domain wipeout* terdeteksi, algoritma dapat segera melakukan *backtrack* tanpa perlu mengeksplorasi cabang-cabang yang dipastikan tidak produktif. Dengan demikian, *forward checking* mengurangi kedalaman pohon pencarian yang perlu dijelajahi sebelum kegagalan ditemukan.

Dalam konteks *Colored Queens*, *forward checking* bekerja sebagai berikut: ketika sebuah menteri ditempatkan pada posisi (r, c) di sektor warna tertentu, algoritma segera memindai domain seluruh sektor warna yang belum diproses dan menghapus setiap posisi yang berada pada baris r , kolom c , atau sel-sel yang bertetangga (*adjacent*) dengan (r, c) . Mengingat *constraint graph* pada *Colored Queens* bersifat *complete* sebagaimana dibahas pada Bagian 2.3.3, setiap penugasan memengaruhi domain seluruh variabel lainnya, sehingga *forward checking* memberikan dampak reduksi yang substansial pada setiap langkah.

Namun, *forward checking* memiliki keterbatasan fundamental: teknik ini hanya memeriksa konsistensi antara variabel yang baru ditugaskan dengan variabel-variabel yang belum ditugaskan, tanpa memeriksa konsistensi antarsesama variabel yang belum ditugaskan [3]. Sebagai ilustrasi, misalkan setelah *forward checking*, domain sektor warna A hanya menyisakan satu posisi di baris 3, sementara domain sektor warna B juga hanya menyisakan satu posisi di baris 3. Kedua domain ini saling bertentangan karena melanggar kendala baris, namun *forward checking* tidak akan mendeteksi konflik tersebut karena keduanya masih merupakan variabel yang belum ditugaskan. Konflik baru akan ditemukan ketika salah satu dari kedua variabel tersebut diproses, yang berpotensi membuang banyak langkah pencarian. Keterbatasan inilah yang diatasi oleh teknik *arc consistency* yang dibahas pada bagian berikutnya.

1 Arc Consistency (AC-3)

2 *Arc consistency* adalah properti konsistensi yang lebih kuat dibandingkan yang ditegakkan oleh
 3 *forward checking*. Sebuah *arc* (x_i, x_j) dalam CSP dikatakan *arc-consistent* apabila untuk setiap
 4 nilai $v \in D_i$, terdapat setidaknya satu nilai $w \in D_j$ sedemikian sehingga penugasan $x_i = v$ dan
 5 $x_j = w$ memenuhi semua kendala di antara x_i dan x_j [12]. Nilai w tersebut disebut sebagai *support*
 6 untuk v pada *arc* (x_i, x_j) . Apabila suatu nilai $v \in D_i$ tidak memiliki *support* pada D_j , maka v
 7 dapat dihapus dari D_i tanpa menghilangkan solusi valid manapun, karena v dipastikan tidak dapat
 8 menjadi bagian dari solusi selama domain x_j tetap seperti saat ini.



Gambar 2.4: Contoh propagasi AC-3 pada *constraint graph* dengan lima variabel. Sebelum propagasi (a), domain variabel masih lebar. Setelah AC-3 diterapkan (b), nilai-nilai yang tidak memiliki *support* dieliminasi, menghasilkan domain yang jauh lebih kecil.

9 Sebuah CSP dikatakan *arc-consistent* secara keseluruhan apabila seluruh *arc* di dalamnya bersifat
 10 *arc-consistent*. Perbedaan utama dengan *forward checking* terletak pada cakupan pemeriksaan:
 11 *forward checking* hanya memeriksa konsistensi antara variabel yang baru ditugaskan dengan variabel
 12 lain, sedangkan *arc consistency* memeriksa konsistensi antara *seluruh* pasangan variabel, termasuk
 13 antarsesama variabel yang belum ditugaskan [3].

14 Algoritma AC-3 (*Arc Consistency Algorithm #3*), yang diperkenalkan oleh Mackworth [12],
 15 merupakan algoritma standar untuk menegakkan *arc consistency*. AC-3 mengelola sebuah antrean
 16 Q yang berisi seluruh *arc* yang perlu diperiksa konsistensinya. Untuk setiap *arc* (x_i, x_j) yang
 17 diambil dari antrean, fungsi REVISE memeriksa apakah ada nilai dalam D_i yang tidak memiliki
 18 *support* di D_j ; nilai-nilai tanpa *support* dihapus dari D_i . Apabila D_i berubah (ada penghapusan),
 19 maka seluruh *arc* (x_k, x_i) untuk setiap tetangga x_k dari x_i (selain x_j) ditambahkan kembali ke
 20 antrean, karena penghapusan nilai dari D_i berpotensi membuat beberapa nilai di D_k kehilangan
 21 *support*-nya. Proses berlanjut hingga antrean kosong (*arc consistency* tercapai) atau domain suatu
 22 variabel menjadi kosong (inkonsistensi terdeteksi). Pseudocode AC-3 diperlihatkan pada Listing 3.

Algorithm 3 Algoritma AC-3

```

1: function AC3(csp)
2:    $Q \leftarrow$  seluruh busur  $(X_i, X_j)$  dalam csp
3:   while  $Q$  tidak kosong do
4:      $(X_i, X_j) \leftarrow \text{REMOVEFROMQUEUE}(Q)$ 
5:     if REVISE(csp,  $X_i, X_j$ ) then
6:       if  $D_{X_i}$  kosong then
7:         return false
8:       end if
9:       for setiap  $X_k$  tetangga  $X_i, X_k \neq X_j$  do
10:        Tambahkan  $(X_k, X_i)$  ke  $Q$ 
11:      end for
12:    end if
13:  end while
14:  return true
15: end function

16: function REVISE(csp,  $X_i, X_j$ )
17:   revised  $\leftarrow$  false
18:   for setiap  $v \in D_{X_i}$  do
19:     if tidak ada nilai  $w \in D_{X_j}$  yang memenuhi kendala  $(X_i, X_j)$  then
20:       Hapus  $v$  dari  $D_{X_i}$ 
21:       revised  $\leftarrow$  true
22:     end if
23:   end for
24:   return revised
25: end function

```

1 Kompleksitas waktu AC-3 adalah $O(cd^3)$, di mana c adalah jumlah kendala (*arc*) dan d adalah
2 ukuran maksimum domain [12]. Analisisnya adalah sebagai berikut: setiap *arc* dapat dimasukkan
3 ke dalam antrian paling banyak d kali (karena setiap kali *arc* diproses ulang, setidaknya satu nilai
4 telah dihapus dari domain terkait), dan setiap pemanggilan fungsi REVISE memerlukan waktu $O(d^2)$
5 untuk memeriksa setiap pasangan nilai. Meskipun terdapat algoritma yang lebih efisien seperti
6 AC-4 dengan kompleksitas optimal $O(cd^2)$ [13], AC-3 tetap menjadi pilihan yang paling banyak
7 digunakan dalam praktik karena kesederhanaan implementasinya dan *overhead* per-operasi yang
8 rendah.

9 Dalam konteks penyelesaian CSP berbasis *backtracking*, AC-3 digunakan sebagai teknik propagasi
10 yang dijalankan setelah *forward checking* pada setiap langkah penugasan. Pendekatan gabungan
11 ini, yang sering disebut *Maintaining Arc Consistency* (MAC), bekerja sebagai berikut [14]: setelah
12 sebuah variabel diberi nilai dan *forward checking* menghapus nilai-nilai yang secara langsung tidak
13 konsisten dari domain variabel-variabel yang belum ditugaskan, AC-3 dijalankan pada seluruh *arc* di
14 antara variabel-variabel yang belum ditugaskan untuk mendeteksi dan mengeliminasi inkonsistensi
15 transitif yang tidak terdeteksi oleh *forward checking*. Apabila proses propagasi ini menghasilkan
16 *domain wipeout* pada salah satu variabel, *backtrack* dilakukan segera. Kombinasi *backtracking* +
17 *forward checking* + AC-3 inilah yang menjadi fondasi solver sistematis pada tugas akhir ini dan
18 implementasinya akan dibahas secara rinci pada Bab ??.

2.4.2 Metaheuristic

Konsep Umum Metaheuristic

Metaheuristic adalah kerangka kerja algoritma tingkat tinggi yang dirancang untuk memandu proses pencarian solusi pada masalah optimasi yang ruang pencariannya terlalu besar atau terlalu kompleks untuk dijelajahi secara sistematis [15]. Istilah ini berasal dari gabungan kata Yunani *meta* (“di atas”) dan *heuriskein* (“menemukan”), yang secara harfiah berarti “strategi pencarian tingkat tinggi.” Berbeda dengan algoritma sistematis yang dibahas pada Bagian 2.4.1, *metaheuristic* bersifat *incomplete*: tidak ada jaminan bahwa solusi optimal — atau bahkan solusi valid sekalipun — akan selalu ditemukan. Sebagai gantinya, *metaheuristic* menawarkan kemampuan untuk menavigasi ruang solusi yang sangat besar secara efisien, dengan harapan menemukan solusi yang cukup baik dalam waktu yang jauh lebih singkat dibandingkan pencarian eksak.

Karakteristik utama yang membedakan *metaheuristic* dari pencarian lokal sederhana (*simple local search*) adalah adanya mekanisme untuk menghindari jebakan *local optimum* [15]. Pencarian lokal sederhana, seperti *hill climbing*, rentan terjebak pada solusi yang optimal secara lokal tetapi bukan optimal secara global, karena algoritma berhenti begitu tidak ada tetangga yang lebih baik dari solusi saat ini. *Metaheuristic* mengatasi keterbatasan ini melalui berbagai mekanisme, antara lain penerimaan solusi yang sementara lebih buruk (seperti pada *Simulated Annealing*), penggunaan memori terhadap solusi-solusi yang telah dikunjungi (seperti pada *Tabu Search*), atau pemanfaatan populasi solusi yang berinteraksi satu sama lain (seperti pada *Genetic Algorithm* dan *Particle Swarm Optimization*).

Secara umum, *metaheuristic* dapat diklasifikasikan berdasarkan beberapa dimensi [15]:

- Berbasis trajektori vs berbasis populasi.** Algoritma berbasis trajektori, seperti *Simulated Annealing* dan *Tabu Search*, bekerja dengan satu solusi yang dimodifikasi secara iteratif. Algoritma berbasis populasi, seperti *Genetic Algorithm* (GA) dan *Particle Swarm Optimization* (PSO), bekerja dengan sekumpulan solusi yang berevolusi secara paralel. Pendekatan berbasis populasi memiliki keunggulan natural dalam hal eksplorasi karena memelihara diversitas solusi di dalam populasi.
- Eksplorasi vs eksploitasi.** Setiap *metaheuristic* harus menyeimbangkan dua kecenderungan yang saling berlawanan: *eksplorasi* (menjelajahi area baru dalam ruang solusi) dan *eksploitasi* (mengintensifkan pencarian di sekitar solusi-solusi terbaik yang telah ditemukan). Terlalu banyak eksplorasi menyebabkan pencarian menjadi acak dan tidak efisien, sementara terlalu banyak eksploitasi menyebabkan konvergensi prematur ke *local optimum*.
- Terinspirasi alam vs non-terinspirasi alam.** Banyak *metaheuristic* terinspirasi dari fenomena biologis atau fisika, seperti evolusi genetik (GA), perilaku kawanan (PSO), dan pendinginan logam (*Simulated Annealing*). Meskipun inspirasi alam memberikan intuisi yang berguna, efektivitas algoritma pada akhirnya bergantung pada mekanisme komputasional yang mendasarinya, bukan pada kesetiaan terhadap analogi biologis.

Dalam konteks penyelesaian *Constraint Satisfaction Problem* (CSP), penggunaan *metaheuristic* memerlukan reformulasi masalah dari paradigma “cari penugasan yang memenuhi seluruh kendala” menjadi paradigma “minimisasi jumlah pelanggaran kendala” [16]. Dengan kata lain, CSP ditransformasikan menjadi *Constraint Optimization Problem* (COP) di mana fungsi objektif (*fitness*

function) mengukur seberapa banyak kendala yang dilanggar oleh suatu solusi kandidat. Solusi valid dalam CSP asli berkorespondensi dengan solusi yang memiliki nilai *fitness* nol — yaitu, tidak ada kendala yang dilanggar. Reformulasi ini memungkinkan penerapan *metaheuristic* yang pada awalnya dirancang untuk masalah optimasi kontinu ke permasalahan kombinatorik diskrit seperti *Colored Queens*, meskipun diperlukan adaptasi representasi solusi dan operator pencarian sebagaimana akan dibahas pada bagian berikutnya.

Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) adalah algoritma optimasi berbasis populasi yang diperkenalkan oleh Kennedy dan Eberhart pada tahun 1995 [17]. Algoritma ini terinspirasi dari perilaku sosial kawanan burung (*bird flocking*) atau gerombolan ikan (*fish schooling*) dalam mencari sumber makanan. Dalam PSO, setiap solusi kandidat disebut *particle* (partikel), dan kumpulan partikel-partikel tersebut disebut *swarm* (kawanan). Setiap partikel menavigasi ruang solusi dengan kecepatan (*velocity*) yang disesuaikan berdasarkan pengalaman individu dan pengalaman sosial dari kawanan.

PSO pada Ruang Kontinu. Dalam formulasi standar untuk ruang kontinu, setiap partikel i memiliki dua atribut utama: posisi $\vec{X}_i$ yang merepresentasikan solusi kandidat dalam ruang pencarian berdimensi d , dan *velocity* $\vec{V}_i$ yang menentukan arah dan jarak perpindahan partikel pada setiap iterasi. Selain itu, setiap partikel menyimpan *personal best* ($pBest_i$), yaitu posisi terbaik yang pernah dicapai oleh partikel tersebut selama proses pencarian, serta mengacu pada posisi terbaik yang dicapai oleh tetangga-tetangganya, yang disebut *neighborhood best* ($nBest$) pada topologi lokal atau *global best* ($gBest$) pada topologi global [17].

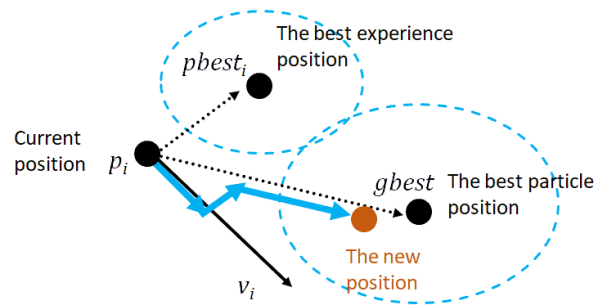
Pada setiap iterasi, *velocity* dan posisi setiap partikel diperbarui menggunakan persamaan berikut [17]:

$$\vec{V}_i^{k+1} = \omega \cdot \vec{V}_i^k + c_1 \cdot r_1 \cdot (pBest_i - \vec{X}_i^k) + c_2 \cdot r_2 \cdot (nBest - \vec{X}_i^k) \quad (2.1)$$

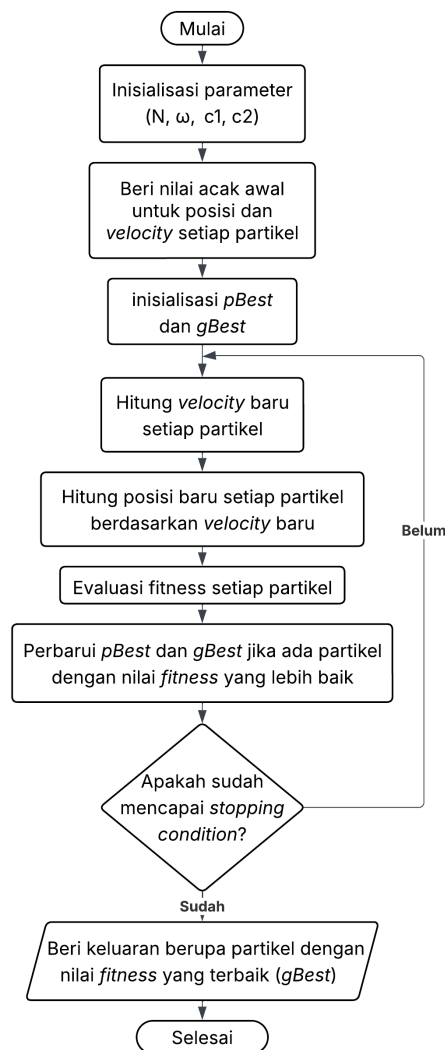
$$\vec{X}_i^{k+1} = \vec{X}_i^k + \vec{V}_i^{k+1} \quad (2.2)$$

di mana $\vec{V}_i^k$ adalah *velocity* partikel i pada iterasi ke- k , $\vec{X}_i^k$ adalah posisi partikel i pada iterasi ke- k , ω adalah *inertia weight* yang mengontrol momentum partikel, c_1 adalah koefisien kognitif yang mengontrol pengaruh *personal best*, c_2 adalah koefisien sosial yang mengontrol pengaruh *neighborhood best*, serta r_1 dan r_2 adalah bilangan acak dalam rentang $[0, 1]$ yang memberikan komponen stokastik pada pencarian.

Persamaan 2.1 menggabungkan tiga komponen pergerakan: (1) *komponen inersia* ($\omega \cdot \vec{V}_i^k$), yaitu kecenderungan partikel untuk terus bergerak ke arah yang sama dengan langkah sebelumnya; (2) *komponen kognitif* ($c_1 \cdot r_1 \cdot (pBest_i - \vec{X}_i^k)$), yaitu tarikan menuju posisi terbaik yang pernah ditemukan oleh partikel itu sendiri; dan (3) *komponen sosial* ($c_2 \cdot r_2 \cdot (nBest - \vec{X}_i^k)$), yaitu tarikan menuju posisi terbaik yang ditemukan oleh tetangga partikel. Interaksi ketiga komponen ini menghasilkan keseimbangan antara eksplorasi dan eksploitasi: komponen inersia mendorong eksplorasi area baru, sedangkan komponen kognitif dan sosial mendorong eksploitasi area yang menjanjikan.



Gambar 2.5: Ilustrasi perpindahan satu partikel berdasarkan *velocity* baru yang dipengaruhi oleh *personal best* dan *global best*.



Gambar 2.6: Diagram alir algoritma PSO.

- 1 **Topologi Kawan.** Topologi kawan menentukan bagaimana informasi mengenai posisi terbaik
- 2 dibagikan di antara partikel-partikel dalam *swarm*. Terdapat dua jenis topologi utama [17]:
- 3 1. **Global best (gBest):** Seluruh partikel dalam *swarm* berbagi informasi tentang satu posisi
- 4 terbaik global. Topologi ini mendorong konvergensi cepat karena seluruh partikel tertarik ke

satu titik terbaik, namun rentan terhadap konvergensi prematur ke *local optimum*.

2. **Local best (lBest)**: Setiap partikel hanya berbagi informasi dengan sekelompok tetangga (*neighborhood*). Informasi terbaik menyebar secara bertahap melalui *neighborhood* yang saling tumpang-tindih. Topologi ini memperlambat konvergensi namun meningkatkan diversitas pencarian dan mengurangi risiko terjebak pada *local optimum*.

Adaptasi PSO untuk Ruang Diskrit. PSO pada awalnya dirancang untuk masalah optimasi dengan ruang solusi kontinu, di mana operasi aritmetika seperti penjumlahan, pengurangan, dan perkalian skalar pada vektor posisi dan *velocity* memiliki interpretasi geometris yang jelas. Namun, pada masalah kombinatorik diskrit seperti *Colored Queens*, ruang solusi terdiri dari kombinasi indeks-indeks diskrit yang tidak dapat dijumlahkan atau dikalikan secara langsung [18].

Kennedy dan Eberhart [19] pertama kali mengadaptasi PSO untuk ruang diskrit biner dengan memperkenalkan konsep *velocity* sebagai probabilitas. Dalam *Binary PSO* (BPSO), *velocity* dihitung menggunakan persamaan standar PSO (Persamaan 2.1), namun hasilnya ditransformasikan menjadi nilai probabilitas dalam rentang $[0, 1]$ menggunakan fungsi *sigmoid*. Probabilitas ini kemudian menentukan apakah suatu dimensi posisi partikel akan berubah atau tidak: apabila bilangan acak yang dihasilkan lebih kecil dari probabilitas tersebut, nilai pada dimensi itu diubah.

Hu, Eberhart, dan Shi [18] mengembangkan gagasan ini lebih lanjut untuk masalah permutasi, khususnya masalah N-Queens. Dalam adaptasi mereka, *velocity* dinormalisasi ke rentang $[0, 1]$ dengan membaginya terhadap nilai maksimum pada partikel. Setiap dimensi kemudian secara acak menentukan apakah sebuah pertukaran (*swap*) perlu dilakukan berdasarkan probabilitas yang ditentukan oleh *velocity*. Apabila pertukaran diperlukan, posisi pada dimensi tersebut diatur agar sesuai dengan posisi pada dimensi yang sama di *nBest*. Pendekatan ini mengubah interpretasi *velocity* dari “jarak perpindahan” pada ruang kontinu menjadi “kemungkinan untuk mengadopsi posisi terbaik yang diketahui” pada ruang diskrit.

Secara umum, adaptasi PSO untuk ruang diskrit memerlukan tiga modifikasi utama:

1. **Representasi posisi**: Posisi partikel direpresentasikan sebagai vektor indeks diskrit, bukan vektor bilangan real. Setiap dimensi vektor merepresentasikan pilihan dari sebuah domain diskrit yang berhingga.
2. **Reinterpretasi velocity**: *Velocity* tidak lagi berarti perpindahan aritmetika, melainkan probabilitas perubahan pada setiap dimensi. Komponen kognitif dan sosial dalam persamaan *velocity* dihitung berdasarkan perbedaan biner (*match/mismatch*) antara posisi saat ini dengan *pBest* dan *nBest*, bukan selisih numerik.
3. **Mekanisme pembaruan posisi**: Posisi diperbarui bukan dengan penjumlahan vektor, melainkan dengan mekanisme probabilistik: untuk setiap dimensi, apabila bilangan acak lebih kecil dari *velocity* pada dimensi tersebut, posisi diubah mengikuti *nBest*; sebaliknya, posisi dipertahankan.

Adaptasi-adaptasi ini memungkinkan PSO diterapkan pada permasalahan kombinatorik diskrit seperti *Colored Queens* tanpa mengubah struktur dasar algoritma. Detail spesifik mengenai bagaimana adaptasi ini diimplementasikan untuk *Colored Queens* akan dibahas pada Bab ??.

2.5 Perbandingan Pendekatan

Bagian ini membandingkan dua kategori pendekatan yang digunakan dalam tugas akhir ini untuk menyelesaikan permasalahan *Colored Queens*: pendekatan sistematis (*complete*) dan pendekatan *metaheuristic* (*incomplete*). Perbandingan dilakukan secara teoretis untuk memberikan landasan bagi analisis empiris yang akan disajikan pada bab-bab selanjutnya.

2.5.1 Algoritma Complete vs Metaheuristic

Perbedaan paling fundamental antara algoritma *complete* dan *metaheuristic* terletak pada jaminan terhadap solusi [3]. Algoritma *complete*, seperti *backtracking* dengan *forward checking* dan AC-3, menjamin bahwa solusi akan ditemukan apabila solusi tersebut ada, atau akan membuktikan ketiadaan solusi apabila tidak ada penugasan yang memenuhi seluruh kendala. Algoritma *metaheuristic*, seperti PSO, tidak memberikan jaminan tersebut: algoritma dapat berakhir tanpa menemukan solusi valid meskipun solusi tersebut ada, atau dapat menemukan solusi yang mendekati optimal tetapi bukan solusi terbaik.

Dari segi kompleksitas waktu, algoritma *complete* memiliki kompleksitas terburuk yang eksponensial terhadap jumlah variabel, yaitu $O(d^n)$ di mana d adalah ukuran domain maksimum dan n adalah jumlah variabel [10]. Meskipun teknik propagasi kendala seperti *forward checking* dan AC-3 dapat mengurangi jumlah simpul yang dieksplorasi secara signifikan, kompleksitas terburuk tetap eksponensial. Di sisi lain, kompleksitas waktu *metaheuristic* bergantung pada parameter yang ditetapkan pengguna (jumlah partikel, jumlah iterasi), sehingga waktu eksekusi dapat dikontrol secara langsung, meskipun dengan *trade-off* terhadap kualitas solusi [15].

Dari segi penggunaan memori, algoritma *backtracking* bekerja pada satu cabang pencarian dalam satu waktu sehingga hanya memerlukan ruang polinomial $O(nd)$ untuk menyimpan domain dan penugasan saat ini [11]. PSO memerlukan ruang yang proporsional terhadap jumlah partikel dikali dimensi masalah, yaitu $O(P \cdot n)$ di mana P adalah jumlah partikel, yang pada umumnya juga bersifat polinomial namun dengan konstanta yang lebih besar.

Tabel 2.1 merangkum perbandingan kedua kategori pendekatan.

Tabel 2.1: Perbandingan algoritma *complete* dan *metaheuristic*

Aspek	Complete (Backtracking+FC+AC-3)	Metaheuristic (PSO)
Jaminan solusi	Ya — menemukan solusi jika ada, membuktikan ketiadaan jika tidak ada	Tidak — dapat gagal menemukan solusi yang ada
Kompleksitas waktu	Eksponensial terburuk $O(d^n)$, namun propagasi kendala mereduksi secara signifikan	Terkontrol oleh parameter (iterasi $\times$ partikel), namun tanpa jaminan kualitas
Penggunaan memori	Polinomial $O(nd)$	Polinomial $O(Pn)$
Skalabilitas	Menurun pada instansi besar akibat ledakan kombinatorik	Lebih stabil, namun kualitas solusi menurun
Determinisme	Deterministik (hasil konsisten)	Stokastik (hasil bervariasi antar eksekusi)

2.5.2 Backtracking+FC+AC-3 vs PSO untuk Colored Queens

Ketika diterapkan secara spesifik pada permasalahan *Colored Queens*, masing-masing pendekatan memiliki karakteristik yang perlu dipertimbangkan.

Pendekatan *backtracking* dengan *forward checking* dan AC-3 memanfaatkan struktur kendala *Colored Queens* secara langsung. Setiap penugasan menteri pada suatu sektor warna segera memicu propagasi kendala ke seluruh sektor warna lainnya, menghapus posisi-posisi yang melanggar kendala baris, kolom, atau *adjacency*. Karena *constraint graph* pada *Colored Queens* bersifat *complete* (sebagaimana dibahas pada Bagian 2.3.3), setiap penugasan memiliki dampak propagasi yang luas, yang memungkinkan pemangkasan domain secara agresif. Keunggulan pendekatan ini adalah kemampuannya untuk mendeteksi inkonsistensi secara dini dan menghindari eksplorasi cabang-cabang yang tidak produktif. Kelemahannya adalah pada papan berukuran besar, jumlah kombinasi yang harus dieksplorasi tetap dapat menjadi sangat besar meskipun dengan propagasi kendala.

Pendekatan PSO beroperasi dengan paradigma yang berbeda. Alih-alih membangun solusi secara inkremental, PSO memulai dengan populasi solusi kandidat yang lengkap (setiap partikel merupakan penugasan untuk seluruh sektor warna) dan secara iteratif memperbaikinya melalui mekanisme *velocity* dan interaksi sosial. Fungsi *fitness* mengukur jumlah pelanggaran kendala, dan algoritma berusaha meminimalkan nilai ini hingga mencapai nol (solusi valid). Keunggulan pendekatan ini adalah kemampuannya untuk menjelajahi banyak area ruang solusi secara paralel melalui populasi partikel. Kelemahannya adalah tidak adanya jaminan konvergensi ke solusi valid, terutama pada masalah dengan ruang solusi yang sangat terkendala (*highly constrained*) di mana solusi valid mungkin hanya menempati sebagian kecil dari ruang pencarian.

Perbedaan mendasar lainnya terletak pada cara kedua pendekatan menangani kendala. Pendekatan *backtracking*+FC+AC-3 menangani kendala secara eksplisit: kendala digunakan untuk memangkas domain dan mencegah penugasan yang tidak konsisten sejak awal. PSO menangani kendala secara implisit melalui fungsi *fitness*: pelanggaran kendala tidak dicegah, melainkan dihukum (*penalized*) dalam evaluasi *fitness*, dan algoritma mengandalkan tekanan selektif untuk mengarahkan pencarian menuju solusi tanpa pelanggaran.

2.5.3 Justifikasi Pemilihan Metode

Pemilihan *backtracking* dengan *forward checking* dan AC-3 sebagai pendekatan sistematis didasarkan pada beberapa pertimbangan. Pertama, kombinasi ketiga teknik ini merupakan pendekatan standar yang telah terbukti efektif untuk berbagai permasalahan CSP [14], sehingga memberikan *baseline* yang kuat untuk perbandingan. Kedua, *forward checking* dan AC-3 saling melengkapi: *forward checking* menangani konsistensi antara variabel yang baru ditugaskan dengan variabel lain, sementara AC-3 menangani konsistensi antarsesama variabel yang belum ditugaskan. Ketiga, jaminan kelengkapan memastikan bahwa kegagalan menemukan solusi dapat diinterpretasikan sebagai bukti ketiadaan solusi, bukan sebagai keterbatasan algoritma.

Pemilihan PSO sebagai pendekatan *metaheuristic* didasarkan pada pertimbangan berikut. Pertama, PSO memiliki struktur yang relatif sederhana dengan sedikit parameter dibandingkan *metaheuristic* lainnya seperti *Genetic Algorithm* yang memerlukan operator *crossover* dan *mutation* [17]. Kedua, adaptasi PSO untuk masalah diskrit telah menunjukkan hasil yang menjanjikan pada masalah N-Queens tradisional [18], sehingga menarik untuk mengevaluasi efektivitasnya pada

1 varian *Colored Queens* yang memiliki kendala tambahan. Ketiga, topologi *local best* (lBest) yang
2 digunakan memberikan mekanisme natural untuk menjaga diversitas populasi dan mengurangi
3 risiko konvergensi prematur, yang merupakan pertimbangan penting pada masalah dengan banyak
4 *local optima*.

5 Perbandingan kedua pendekatan ini diharapkan memberikan wawasan mengenai *trade-off* antara
6 kelengkapan solusi dan efisiensi waktu pada permasalahan *Colored Queens* dengan berbagai ukuran
7 papan, mulai dari ukuran kecil (7×7) hingga besar (30×30). Hasil pengujian dan analisis empiris
8 akan disajikan pada Bab ??.

BAB 3

METODOLOGI PENELITIAN

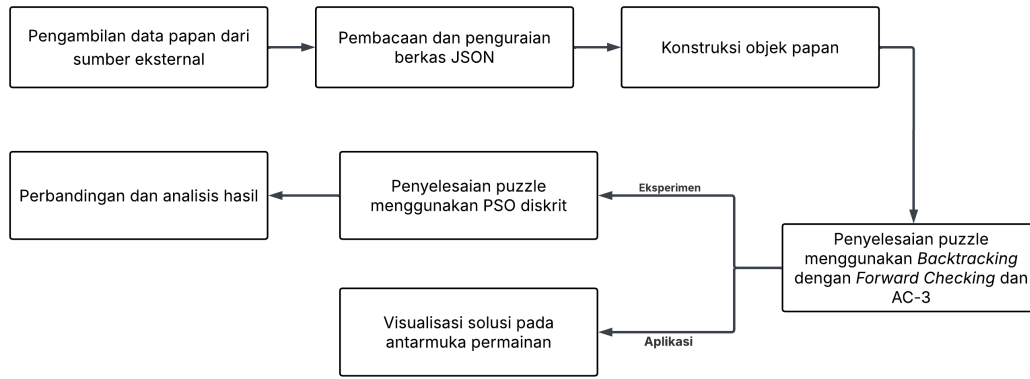
3.1 Gambaran Umum Penelitian

3.2 Gambaran Umum Penelitian

Penelitian ini bertujuan untuk merancang, mengimplementasikan, dan membandingkan dua pendekatan algoritmik dalam menyelesaikan permasalahan *Colored Queens*: pendekatan berbasis pencarian sistematis menggunakan *Backtracking* yang diperkuat dengan *Forward Checking* dan AC-3 sebagaimana dibahas pada Bagian ??, serta pendekatan berbasis *metaheuristic* menggunakan *Particle Swarm Optimization* (PSO) diskrit sebagaimana dibahas pada Bagian ?. Selain kedua solver tersebut, penelitian ini juga mencakup perancangan dan implementasi sebuah aplikasi permainan berbasis antarmuka grafis yang berfungsi sebagai platform visualisasi dan pengujian.

Secara garis besar, sistem yang dibangun terdiri atas tiga komponen utama. Pertama, modul *data loader* yang bertugas membaca dan mengurai berkas puzzle dalam format JSON ke dalam struktur papan yang dapat diproses oleh solver. Kedua, modul solver yang mengimplementasikan kedua algoritma pencarian secara terpisah dan independen. Ketiga, modul antarmuka pengguna yang dibangun menggunakan Java Swing dan JFrame, yang menampilkan papan permainan secara visual serta memungkinkan pengguna untuk berinteraksi secara langsung maupun mengaktifkan solver secara otomatis.

Dalam konteks aplikasi permainan, solver yang digunakan secara eksklusif adalah *backtracking* dengan *forward checking* dan AC-3, yang menjamin ditemukannya solusi dan dijalankan secara asinkron di latar belakang saat pengguna membuka sebuah puzzle. Solver PSO diskrit tidak diintegrasikan ke dalam aplikasi permainan, melainkan digunakan secara terpisah dalam kerangka eksperimen untuk dibandingkan dengan solver *backtracking* dari segi waktu eksekusi, tingkat keberhasilan, dan jumlah iterasi. Alur kerja sistem secara keseluruhan, baik alur aplikasi maupun alur eksperimen, diilustrasikan pada Gambar 3.1.



Gambar 3.1: Alur kerja sistem secara keseluruhan.

Berikut adalah penjelasan setiap proses pada Gambar 3.1:

1. **Pengambilan data papan dari sumber eksternal.** Data puzzle diperoleh dari situs *playqueensgame.com* melalui proses *web scraping* menggunakan *Selenium WebDriver* dengan bahasa Python. Untuk setiap level, skrip mengakses halaman puzzle dan mengekstrak atribut **data-row**, **data-col**, serta properti CSS **background-color** dari setiap elemen sel pada *Document Object Model (DOM)*. Data yang diekstrak kemudian disimpan dalam format JSON dengan struktur satu objek per sel yang memuat atribut **row**, **col**, dan **color**. Proses ini menghasilkan total 1.502 berkas JSON yang mencakup 250 level untuk setiap ukuran papan standar (7×7 hingga 12×12) serta masing-masing 1 level untuk ukuran 20×20 dan 30×30 . Rincian dataset disajikan pada Bagian 3.7.1.
2. **Pembacaan dan penguraian berkas JSON.** Kelas **BoardImporter** membaca berkas JSON dan mengonversi setiap entri menjadi objek **Cell** yang menyimpan koordinat baris, kolom, dan nilai warna RGB. Nilai warna yang semula dalam format **rgba(R, G, B, A)** diuraikan menjadi komponen integer terpisah. Penjelasan lebih lanjut mengenai representasi sel dan proses impor disajikan pada Bagian 3.3.1.
3. **Konstruksi objek papan.** Seluruh objek **Cell** diserahkan kepada kelas **Board**, yang mengelompokkan sel-sel berdasarkan warna ke dalam pemetaan **LinkedHashMap<String, List<int[]>**. Hasil dari proses ini adalah sebuah struktur data yang merepresentasikan papan secara keseluruhan dan siap digunakan oleh solver maupun antarmuka. Representasi papan dibahas pada Bagian 3.3.1, sedangkan pemodelan sebagai CSP dibahas pada Bagian 3.3.
4. **Penyelesaian puzzle menggunakan Backtracking dengan Forward Checking dan AC-3.** Solver memodelkan papan sebagai *Constraint Satisfaction Problem* dan melakukan pencarian sistematis dengan pemangkasan domain. Pada alur aplikasi, solver ini dijalankan secara asinkron dan menghasilkan solusi yang digunakan untuk fitur *hint* dan *auto-solve*. Pada alur eksperimen, solver ini menghasilkan metrik waktu eksekusi dan jumlah langkah. Perancangan solver dibahas pada Bagian 3.4.
5. **Penyelesaian puzzle menggunakan PSO diskrit** (khusus alur eksperimen). Solver berbasis populasi mengeksplorasi ruang solusi melalui sejumlah partikel yang berinteraksi dalam *neighborhood*. Solver ini menghasilkan metrik waktu eksekusi, tingkat keberhasilan, dan nilai *fitness* akhir untuk keperluan perbandingan. Perancangan solver dibahas pada Bagian

3.5.

6. **Visualisasi solusi pada antarmuka permainan** (khusus alur aplikasi). Solusi berupa pasangan koordinat penempatan menteri dirender pada papan berwarna melalui kelas `GameWindow`, yang juga mengelola interaksi pengguna seperti penempatan manual, validasi *real-time*, dan fitur *hint*. Perancangan antarmuka dibahas pada Bagian 3.6.3.
7. **Perbandingan dan analisis hasil** (khusus alur eksperimen). Metrik dari kedua solver diintegrasikan dan dibandingkan berdasarkan waktu eksekusi, tingkat keberhasilan, dan karakteristik pencarian pada berbagai ukuran papan dan konfigurasi parameter. Skenario pengujian dibahas pada Bagian 3.7 dan hasil analisis disajikan pada Bab ??.

3.3 Pemodelan Permasalahan *Colored Queens* sebagai CSP

Sebagaimana telah dibahas pada Bagian 2.3, sebuah *Constraint Satisfaction Problem* didefinisikan oleh tiga komponen utama: himpunan variabel, domain masing-masing variabel, dan himpunan kendala yang harus dipenuhi secara bersamaan [3]. Bagian ini menjabarkan secara konkret bagaimana ketiga komponen tersebut didefinisikan dalam konteks permasalahan *Colored Queens*, sekaligus menjelaskan bagaimana representasi tersebut diimplementasikan dalam sistem yang dibangun.

3.3.1 Representasi Papan

Papan permainan *Colored Queens* dimodelkan sebagai sebuah grid berukuran $n \times n$, di mana n menyatakan jumlah baris, jumlah kolom, sekaligus jumlah sektor warna yang ada pada papan. Setiap sel pada papan diidentifikasi oleh pasangan koordinat (*baris*, *kolom*) dengan indeks berbasis nol, sehingga sel pojok kiri atas memiliki koordinat $(0, 0)$ dan sel pojok kanan bawah memiliki koordinat $(n - 1, n - 1)$.

Dalam implementasi, setiap sel direpresentasikan oleh objek `Cell` yang menyimpan koordinat baris, koordinat kolom, serta nilai warna dalam format RGB. Informasi warna digunakan sebagai kunci pengelompokan sel ke dalam sektor warna yang bersesuaian. Papan secara keseluruhan direpresentasikan oleh objek `Board`, yang menyimpan pemetaan dari setiap sektor warna ke daftar sel-sel yang termasuk dalam sektor tersebut melalui struktur `LinkedHashMap<String, List<int[]>`. Kunci peta adalah representasi string dari nilai RGB warna dalam format "R,G,B", sedangkan nilainya adalah daftar pasangan koordinat [*baris*, *kolom*] dari seluruh sel yang memiliki warna tersebut. Penggunaan `LinkedHashMap` secara khusus dipilih untuk mempertahankan urutan insersi warna, yang berpengaruh pada urutan pemrosesan variabel oleh solver sebagaimana akan dijelaskan pada Bagian 3.4.4.

Data papan dibaca dari berkas JSON dengan format `{ukuran}x{ukuran}_level{nomor}.json` oleh kelas `BoardImporter`. Setiap entri dalam berkas JSON mewakili satu sel dan memuat atribut `row`, `col`, serta `color` dalam format `rgba(R, G, B, A)`.

3.3.2 Definisi Variabel

Mengacu pada kerangka CSP yang dijelaskan pada Bagian 2.3, variabel dalam permasalahan *Colored Queens* didefinisikan sebagai sektor warna pada papan. Secara formal, diberikan himpunan warna $C = \{c_1, c_2, \dots, c_n\}$, maka himpunan variabel adalah:

$$X = \{X_{c_1}, X_{c_2}, \dots, X_{c_n}\} \quad (3.1)$$

di mana setiap variabel X_{c_i} merepresentasikan pilihan posisi penempatan menteri untuk sektor warna c_i .

Pemilihan sektor warna sebagai variabel—bukan baris atau kolom seperti pada N-Queens klasik—didasarkan pada struktur kendala *Colored Queens* itu sendiri. Aturan permainan mengharuskan tepat satu menteri ditempatkan pada setiap sektor warna (kendala *one queen per color*), sehingga setiap sektor warna secara alami berkorespondensi dengan satu variabel keputusan. Dengan pemodelan ini, himpunan variabel memiliki kardinalitas yang sama dengan ukuran papan n , dan setiap penugasan lengkap (*complete assignment*) secara otomatis memenuhi kendala jumlah menteri per warna.

3.3.3 Definisi Domain

Domain dari setiap variabel X_{c_i} adalah himpunan semua sel yang termasuk dalam sektor warna c_i . Secara formal:

$$D_{c_i} = \{(r, k) \mid \text{sel } (r, k) \text{ memiliki warna } c_i\} \quad (3.2)$$

Dengan demikian, ukuran domain setiap variabel dapat berbeda-beda tergantung pada bentuk dan luas sektor warna yang bersangkutan pada papan. Tidak seperti N-Queens klasik yang menggunakan representasi permutasi dengan domain seragam $\{1, 2, \dots, n\}$ [2], domain pada *Colored Queens* bersifat heterogen dan bergantung pada tata letak papan yang diberikan.

Dalam implementasi, domain setiap variabel direpresentasikan menggunakan dua struktur yang bekerja secara bersama-sama. Daftar sel `List<int[]>` menyimpan koordinat aktual dari seluruh sel dalam domain secara statis sebagai referensi. `BitSet` digunakan untuk merepresentasikan status kevalidan setiap sel secara dinamis selama proses pencarian: bit ke- i bernilai 1 apabila sel ke- i masih valid sebagai kandidat penempatan menteri, dan bernilai 0 apabila sel tersebut telah dipangkas oleh mekanisme *forward checking* atau propagasi AC-3. Kombinasi ini memungkinkan operasi pemangkasan dan pemulihan domain dilakukan dalam waktu $O(1)$ per sel.

3.3.4 Definisi *Constraint*

Sesuai dengan aturan permainan *Colored Queens* yang dijabarkan pada Bagian 2.2.1, terdapat tiga jenis kendala yang harus dipenuhi secara bersamaan oleh setiap pasangan variabel (X_{c_i}, X_{c_j}) dengan $i \neq j$. Ketiga kendala ini bersifat biner dan membentuk *constraint graph* yang lengkap (*complete*), artinya setiap pasang variabel terhubung oleh kendala [14].

Kendala Baris. Dua menteri tidak boleh ditempatkan pada baris yang sama. Untuk dua sel (r_1, k_1) dan (r_2, k_2) yang ditugaskan kepada variabel X_{c_i} dan X_{c_j} , kendala ini dinyatakan sebagai:

$$r_1 \neq r_2 \quad (3.3)$$

Kendala Kolom. Dua menteri tidak boleh ditempatkan pada kolom yang sama:

$$k_1 \neq k_2 \quad (3.4)$$

Kendala *Adjacency*. Berbeda dari permasalahan N-Queens klasik yang melarang konflik diagonal tak terbatas [1], *Colored Queens* hanya melarang penempatan dua menteri pada sel yang bertetangga secara langsung dalam delapan arah (horizontal, vertikal, dan diagonal). Dua sel (r_1, k_1) dan (r_2, k_2) dinyatakan bertetangga apabila:

$$|r_1 - r_2| \leq 1 \quad \text{dan} \quad |k_1 - k_2| \leq 1 \quad (3.5)$$

Perlu dicatat bahwa tidak ada larangan bagi dua menteri untuk berada pada diagonal yang sama, selama keduanya tidak bertetangga secara langsung. Hal ini merupakan perbedaan struktural yang signifikan dari N-Queens klasik dan menjadi karakteristik utama dari *Colored Queens* sebagaimana dibahas pada Bagian 2.2.1.

Dalam implementasi, ketiga kendala di atas diperiksa sekaligus melalui satu fungsi konflik tunggal. Dua sel dinyatakan berkonflik apabila memenuhi salah satu dari kondisi berikut:

$$\text{konflik}((r_1, k_1), (r_2, k_2)) \iff (r_1 = r_2) \vee (k_1 = k_2) \vee (|r_1 - r_2| \leq 1 \wedge |k_1 - k_2| \leq 1) \quad (3.6)$$

3.3.5 Representasi Solusi

Sebuah solusi valid dari permasalahan *Colored Queens* yang dimodelkan sebagai CSP adalah sebuah penugasan lengkap $A = \{X_{c_1} \mapsto s_1, X_{c_2} \mapsto s_2, \dots, X_{c_n} \mapsto s_n\}$ di mana $s_i \in D_{c_i}$ untuk setiap i , dan tidak ada pasangan (s_i, s_j) dengan $i \neq j$ yang melanggar ketiga kendala yang telah didefinisikan pada Bagian 3.3.4.

Dalam implementasi solver *backtracking*, solusi disimpan sebagai array integer `solution[]` berindeks variabel (warna), di mana `solution[i]` menyimpan *indeks* sel terpilih dalam daftar domain warna ke- i . Koordinat aktual sel kemudian dapat diperoleh dengan `colorCells.get(colors.get(i)).get(solution[i])`. Representasi berbasis indeks ini dipilih karena lebih efisien secara memori dan memudahkan operasi pemangkasan domain berbasis `BitSet`.

Untuk keperluan antarmuka, solusi dikonversi ke dalam format matriks dua dimensi `int[][]` `result` berukuran $n \times 2$, di mana `result[i][0]` dan `result[i][1]` masing-masing menyimpan koordinat baris dan kolom menteri untuk warna ke- i . Format ini kemudian digunakan oleh `GameWindow` untuk merender posisi menteri pada tampilan papan.

3.4 Perancangan Algoritma *Backtracking* dengan *Forward Checking* dan AC-3

Bagian ini menjelaskan perancangan dan implementasi algoritma pencarian sistematis yang digunakan untuk menyelesaikan permasalahan *Colored Queens*. Algoritma ini menggabungkan *backtracking* sebagai kerangka pencarian utama dengan dua mekanisme pemangkasan ruang pencarian, yaitu *forward checking* dan AC-3, yang landasan teorinya telah dibahas pada Bagian ???. Penjelasan dimulai

1 dari alur dasar pencarian dan pemeriksaan kendala, dilanjutkan dengan mekanisme pemangkasan
 2 yang menjadi inti efisiensi algoritma.

3 3.4.1 Alur Dasar *Backtracking*

4 Algoritma *backtracking* yang diimplementasikan menggunakan pendekatan rekursif *depth-first* yang
 5 memproses variabel (sektor warna) satu per satu secara berurutan. Prosedur utama pencarian
 6 direpresentasikan oleh fungsi `placeQueens(colorIndex)`, yang bekerja sebagai berikut.

7 Pada setiap pemanggilan rekursif, fungsi menerima indeks warna `colorIndex` yang menunjukkan
 8 variabel mana yang sedang diproses. Apabila indeks telah mencapai jumlah total warna, seluruh
 9 variabel telah berhasil ditugaskan dan solusi dinyatakan ditemukan. Apabila domain variabel
 10 saat ini kosong (tidak ada sel valid yang tersisa), fungsi segera mengembalikan nilai `false` tanpa
 11 melanjutkan pencarian.

12 Untuk setiap sel valid dalam domain variabel saat ini, algoritma melakukan langkah-langkah
 13 berikut: (1) menempatkan menteri pada sel tersebut, (2) menjalankan mekanisme pemangkasan
 14 terhadap domain variabel-variabel yang belum ditugaskan, (3) memeriksa apakah pemangkasan
 15 mengakibatkan domain suatu variabel menjadi kosong, dan (4) melanjutkan pencarian secara
 16 rekursif ke variabel berikutnya apabila tidak ada domain yang kosong. Apabila pencarian rekursif
 17 gagal atau domain ditemukan kosong, algoritma membatalkan penugasan saat ini, memulihkan
 18 seluruh domain yang telah dipangkas, dan mencoba sel berikutnya. Mekanisme pemangkasan dan
 19 pemulihan domain dibahas secara rinci pada Bagian 3.4.3.

20 Pseudocode alur dasar *backtracking* diperlihatkan pada Algoritma 4.

Algorithm 4 Alur dasar *backtracking* untuk *Colored Queens*

```

1: function PLACEQUEENS(colorIndex)
2:   if colorIndex =  $|C|$  then
3:     return true                                     ▷ Seluruh variabel telah ditugaskan
4:   end if
5:   color  $\leftarrow C[\textit{colorIndex}]$ 
6:   if  $|D_{\textit{color}}| = 0$  then
7:     return false                                     ▷ Domain kosong, gagal
8:   end if
9:   for setiap sel s dalam  $D_{\textit{color}}$  yang masih valid do
10:    Tempatkan menteri pada sel s
11:    checkpoint  $\leftarrow$  posisi pruneStack saat ini
12:    Jalankan pemangkasan (forward checking + AC-3)
13:    if tidak ada domain variabel lain yang kosong then
14:      if PLACEQUEENS(colorIndex + 1) then
15:        return true
16:      end if
17:    end if
18:    Pulihkan domain ke checkpoint
19:    Batalkan penempatan menteri pada sel s
20:  end for
21:  return false
22: end function

```

Sistem mencatat dua metrik selama proses pencarian: jumlah langkah (*steps*), yaitu banyaknya penugasan yang berhasil diteruskan ke pemanggilan rekursif berikutnya, dan jumlah *backtracks*, yaitu banyaknya pembatalan penugasan yang terjadi akibat kegagalan pencarian pada tingkat yang lebih dalam.

3.4.2 Pemeriksaan *Constraint*

Pemeriksaan kendala merupakan operasi fundamental yang digunakan di seluruh bagian algoritma, baik pada tahap *forward checking* maupun pada tahap propagasi AC-3. Sebagaimana didefinisikan pada Bagian 3.3.4, dua sel (r_1, k_1) dan (r_2, k_2) dinyatakan berkonflik apabila memenuhi salah satu dari tiga kondisi: berada pada baris yang sama ($r_1 = r_2$), berada pada kolom yang sama ($k_1 = k_2$), atau bertetangga secara langsung ($|r_1 - r_2| \leq 1 \wedge |k_1 - k_2| \leq 1$).

Dalam implementasi, ketiga kondisi ini diperiksa dalam satu fungsi `conflicts()` yang mengembalikan nilai boolean. Pemeriksaan dilakukan secara *inline* tanpa mengonstruksi struktur data terpisah untuk zona serangan, sehingga kompleksitas per pemeriksaan adalah $O(1)$. Perlu dicatat bahwa kondisi ketiga (adjacency) secara implisit mencakup kasus dua menteri yang berada pada sel yang sama, karena selisih baris dan kolom keduanya bernilai nol yang memenuhi syarat ≤ 1 . Namun, kasus ini tidak pernah terjadi dalam praktik karena setiap variabel merepresentasikan sektor warna yang berbeda dan sel-sel antarsektor tidak beririsan.

Berbeda dari permasalahan N-Queens klasik yang memeriksa konflik diagonal tak terbatas melalui kondisi $|i - j| = |\pi_i - \pi_j|$ [1], pemeriksaan pada *Colored Queens* hanya bersifat lokal: dua menteri yang berada pada diagonal yang sama namun berjarak lebih dari satu sel tidak dianggap berkonflik. Perbedaan ini berimplikasi pada tingkat pemangkasan domain—kendala *adjacency* hanya memangkas sel-sel yang bertetangga langsung dengan menteri yang baru ditempatkan, sedangkan kendala baris dan kolom memangkas seluruh baris atau kolom yang bersangkutan.

3.4.3 Mekanisme Pemangkasan: *Forward Checking* dan AC-3

Efisiensi algoritma *backtracking* sangat bergantung pada kemampuannya untuk menghindari eksplorasi cabang pencarian yang pasti gagal sedini mungkin. Dalam implementasi ini, dua mekanisme pemangkasan domain digunakan secara berurutan setelah setiap penugasan variabel: *forward checking* sebagai pemangkasan tahap pertama, dan propagasi AC-3 sebagai pemangkasan tahap kedua yang lebih mendalam. Seluruh operasi pemangkasan dicatat dalam sebuah struktur *pruneStack* agar dapat dibatalkan secara efisien saat algoritma melakukan *backtrack*.

Forward Checking

Setelah sebuah menteri ditempatkan pada sel (r, k) untuk variabel warna ke- i , *forward checking* memeriksa seluruh sel yang masih valid dalam domain setiap variabel warna yang belum ditugaskan $(i + 1, i + 2, \dots, n)$. Untuk setiap sel tersebut, fungsi konflik sebagaimana didefinisikan pada Bagian 3.4.2 dievaluasi terhadap posisi menteri yang baru ditempatkan. Apabila sebuah sel dinyatakan berkonflik, bit yang bersesuaian pada `BitSet` domain warna tersebut di-*clear* dan pencacah ukuran domain (`colorCellCount`) dikurangi. Setiap pemangkasan dicatat sebagai objek `PruneAction` yang disimpan pada *pruneStack*.

Pseudocode tahap *forward checking* diperlihatkan pada Algoritma 5.

Algorithm 5 *Forward checking untuk Colored Queens*

```

1: function FORWARDCHECK(row, col, placedColorIdx)
2:   for colorIdx  $\leftarrow$  placedColorIdx + 1 hingga  $|C| - 1$  do
3:     valid  $\leftarrow$  BitSet domain warna colorIdx
4:     for setiap indeks i yang masih aktif dalam valid do
5:       (r2, k2)  $\leftarrow$  koordinat sel ke-i dari warna colorIdx
6:       if row = r2  $\vee$  col = k2  $\vee$  ( $|row - r_2| \leq 1 \wedge |col - k_2| \leq 1$ ) then
7:         Hapus bit i dari valid
8:         Kurangi colorCellCount[colorIdx]
9:         Catat (colorIdx, i) pada pruneStack
10:      end if
11:    end for
12:  end for
13: end function

```

Dampak *forward checking* bersifat langsung: hanya sel-sel yang berkonflik dengan menteri yang *baru saja* ditempatkan yang dipangkas. Mekanisme ini tidak memperhitungkan interaksi antara domain variabel-variabel yang belum ditugaskan satu sama lain. Konsekuensinya, terdapat kemungkinan sebuah sel tetap berada dalam domain suatu variabel meskipun sel tersebut tidak memiliki *support* yang konsisten di variabel lain. Keterbatasan inilah yang ditangani oleh propagasi AC-3.

Propagasi AC-3

Setelah *forward checking* selesai, algoritma melanjutkan dengan propagasi busur AC-3 untuk menegakkan konsistensi busur antara seluruh pasang variabel yang belum ditugaskan. Sebagaimana dijelaskan pada Bagian ??, AC-3 bekerja dengan memelihara sebuah antrean busur dan memeriksa apakah setiap nilai dalam domain suatu variabel memiliki *support* yang konsisten di variabel lain.

Dalam konteks implementasi ini, antrean awal diinisialisasi dengan seluruh pasang busur (*i, j*) untuk $i \neq j$ di antara variabel-variabel yang belum ditugaskan (indeks $>$ *placedColorIdx*). Untuk setiap busur (*from, to*) yang diambil dari antrean, fungsi **Revise** memeriksa setiap sel yang masih valid dalam domain variabel *to*. Sebuah sel s_{to} dinyatakan *supported* apabila terdapat setidaknya satu sel s_{from} dalam domain variabel *from* yang tidak berkonflik dengan s_{to} . Selain itu, s_{to} juga diperiksa terhadap seluruh menteri yang telah ditempatkan sebelumnya: apabila s_{to} berkonflik dengan salah satu menteri yang sudah ada, maka *support* dibatalkan.

Apabila sebuah sel tidak memiliki *support*, sel tersebut dihapus dari domain dan dicatat pada *pruneStack*. Apabila fungsi **Revise** menghasilkan perubahan (setidaknya satu sel dihapus), busur busur baru ditambahkan ke antrean: untuk setiap variabel *k* yang bukan merupakan variabel *from* dan bukan variabel *to*, busur (*to, k*) ditambahkan agar dampak pemangkasan terpropagasi lebih lanjut. Propagasi berhenti apabila antrean kosong atau domain suatu variabel menjadi kosong.

Pseudocode fungsi **Revise** dan propagasi busur diperlihatkan pada Algoritma 6 dan Algoritma 7.

Algorithm 6 Fungsi *Revise* untuk *Colored Queens*

```

1: function REVISE(fromColorIdx, toColorIdx)
2:   revised  $\leftarrow$  false
3:   for setiap indeks i yang masih aktif dalam  $D_{to}$  do
4:     supported  $\leftarrow$  false
5:     for setiap indeks j yang masih aktif dalam  $D_{from}$  do
6:       if  $\neg$  konflik( $sel_j^{from}$ ,  $sel_i^{to}$ ) then
7:         supported  $\leftarrow$  true
8:         break
9:       end if
10:    end for
11:    if supported then
12:      for setiap variabel p yang telah ditugaskan,  $p \neq toColorIdx$  do
13:        if konflik( $sel_p^{placed}$ ,  $sel_i^{to}$ ) then
14:          supported  $\leftarrow$  false
15:          break
16:        end if
17:      end for
18:    end if
19:    if  $\neg$  supported then
20:      Hapus bit i dari  $D_{to}$ 
21:      Kurangi colorCellCount[toColorIdx]
22:      Catat (toColorIdx, i) pada pruneStack
23:      revised  $\leftarrow$  true
24:    end if
25:  end for
26:  return revised
27: end function

```

Algorithm 7 Propagasi busur AC-3 setelah penugasan

```

1: function PROPAGATEARCS(queue)
2:   while queue tidak kosong do
3:     (from, to)  $\leftarrow$  ambil busur dari queue
4:     if REVISE(from, to) then
5:       if  $|D_{to}| = 0$  then
6:         return ▷ Domain kosong, hentikan propagasi
7:       end if
8:       for setiap variabel k,  $k \neq to$ ,  $k \neq from$  do
9:         Tambahkan busur (to, k) ke queue
10:      end for
11:    end if
12:  end while
13: end function

```

1 Integrasi dan Mekanisme Pemulihan

- 2 Kedua mekanisme pemangkasan bekerja secara berurutan dalam setiap iterasi pencarian. Setelah
- 3 sebuah menteri ditempatkan, *forward checking* terlebih dahulu memangkas sel-sel yang berkonflik
- 4 langsung dengan penempatan tersebut. Selanjutnya, AC-3 mempropagasi dampak pemangkasan

tersebut ke seluruh pasang variabel yang belum ditugaskan, sehingga inkonsistensi yang tidak terdeteksi oleh *forward checking* dapat terungkap. Urutan ini memastikan bahwa AC-3 beroperasi pada domain yang telah dipersempit oleh *forward checking*, sehingga jumlah busur yang perlu diperiksa berkurang.

Seluruh pemangkasan yang dilakukan oleh kedua mekanisme dicatat secara kronologis pada satu *pruneStack* tunggal. Sebelum pemangkasan dimulai, posisi saat ini pada *stack* dicatat sebagai *checkpoint*. Apabila pencarian pada cabang saat ini gagal dan algoritma perlu melakukan *backtrack*, fungsi `undoPrunes` mengembalikan seluruh bit yang telah di-clear sejak *checkpoint* terakhir. Mekanisme ini menjamin bahwa domain setiap variabel dipulihkan ke kondisi tepat sebelum penugasan yang gagal, tanpa memerlukan penyalinan struktur data yang mahal.

Deteksi kegagalan dini dilakukan melalui fungsi `anyColorExhausted`, yang memeriksa apakah terdapat variabel yang belum ditugaskan dengan domain berukuran nol. Pemeriksaan ini dilakukan tepat setelah kedua tahap pemangkasan selesai dan sebelum pemanggilan rekursif berikutnya. Apabila ditemukan domain yang kosong, algoritma langsung melakukan *backtrack* tanpa melanjutkan pencarian, sehingga menghemat komputasi yang tidak produktif.

3.4.4 Strategi Pemilihan Variabel

Urutan pemrosesan variabel dalam algoritma *backtracking* dapat berpengaruh signifikan terhadap efisiensi pencarian. Sebagaimana dibahas pada Bagian ??, heuristik seperti *Minimum Remaining Values* (MRV) memilih variabel dengan domain terkecil terlebih dahulu, dengan tujuan mendeteksi kegagalan sedini mungkin dan mengurangi faktor percabangan pada tingkat awal pohon pencarian [3].

Dalam implementasi ini, urutan pemrosesan variabel tidak ditentukan oleh heuristik dinamis, melainkan mengikuti urutan insersi warna pada struktur `LinkedHashMap` yang dibangun oleh `BoardImporter` saat membaca berkas JSON. Urutan ini bersifat tetap sepanjang eksekusi solver dan ditentukan oleh urutan kemunculan warna dalam berkas data papan. Konsekuensinya, variabel diproses dalam urutan statis yang sama untuk setiap eksekusi pada papan yang sama, sehingga algoritma bersifat deterministik.

Keputusan untuk tidak menggunakan heuristik dinamis seperti MRV didasarkan pada pertimbangan kesederhanaan implementasi. Penerapan MRV memerlukan evaluasi ulang ukuran domain seluruh variabel yang belum ditugaskan pada setiap tingkat rekursi, yang menambahkan *overhead* sebesar $O(n)$ per tingkat. Meskipun *overhead* ini relatif kecil dibandingkan biaya propagasi kendala, manfaatnya pada permasalahan *Colored Queens* tidak selalu signifikan karena ukuran domain antarvariabel pada tahap awal pencarian cenderung tidak jauh berbeda—selisihnya bergantung pada luas sektor warna yang umumnya tidak terlalu bervariasi pada papan berukuran kecil hingga menengah.

Namun demikian, ketiadaan heuristik pemilihan variabel dapat menjadi kelemahan pada papan berukuran besar yang memiliki variasi ukuran sektor warna yang signifikan. Pada kasus tersebut, pemrosesan sektor warna dengan domain besar di awal pencarian dapat menghasilkan faktor percabangan yang tinggi dan memperlambat konvergensi. Potensi peningkatan efisiensi melalui penerapan heuristik pemilihan variabel dicatat sebagai salah satu arah pengembangan pada Bab ??.

3.5 Perancangan Algoritma *Particle Swarm Optimization* Diskrit

Bagian ini menjelaskan perancangan dan implementasi algoritma *Particle Swarm Optimization* (PSO) diskrit yang digunakan sebagai pendekatan *metaheuristic* untuk menyelesaikan permasalahan *Colored Queens*. Landasan teori PSO, termasuk formulasi kontinu asli dan adaptasi diskritnya, telah dibahas pada Bagian ???. Penjelasan pada bagian ini berfokus pada bagaimana konsep-konsep tersebut diadaptasi secara konkret untuk struktur permasalahan *Colored Queens*.

3.5.1 Representasi Partikel

Setiap partikel dalam populasi merepresentasikan satu kandidat solusi lengkap untuk permasalahan *Colored Queens*. Mengacu pada pemodelan CSP yang telah didefinisikan pada Bagian 3.3, sebuah kandidat solusi terdiri atas penugasan posisi menteri untuk setiap sektor warna. Dalam implementasi, kandidat solusi disimpan sebagai array integer `candidate[]` berukuran n (jumlah warna), di mana `candidate[i]` menyimpan indeks sel terpilih dalam daftar domain warna ke- i . Dengan demikian, setiap elemen array merujuk pada sebuah sel spesifik dalam sektor warna yang bersesuaian, dan keseluruhan array merepresentasikan satu konfigurasi penempatan n menteri pada papan.

Selain posisi, setiap partikel juga menyimpan sebuah array *velocity* bertipe `double[]` dengan ukuran yang sama. Berbeda dari PSO kontinu di mana *velocity* menyatakan perpindahan dalam ruang kontinu, *velocity* pada PSO diskrit ini diinterpretasikan sebagai probabilitas perubahan pada setiap dimensi. Nilai $v_j \in [0, 1]$ pada dimensi ke- j menyatakan peluang partikel tersebut mengubah posisinya pada dimensi j dengan mengadopsi posisi terbaik dari *neighborhood*-nya. Interpretasi probabilistik ini mengikuti pendekatan PSO biner yang diusulkan oleh Kennedy dan Eberhart [19], yang diadaptasi untuk domain diskrit berindeks.

Setiap partikel menyimpan dua informasi tambahan yang diperlukan selama proses optimasi: *personal best* (`pBest`), yaitu kandidat solusi terbaik yang pernah ditemukan oleh partikel tersebut sepanjang iterasi, beserta nilai *fitness*-nya (`pBestFitness`). Informasi ini digunakan dalam perhitungan pembaruan *velocity* sebagaimana akan dijelaskan pada Bagian 3.5.5.

3.5.2 Inisialisasi Populasi

Proses inisialisasi terdiri atas tiga tahap yang dieksekusi secara berurutan: pembangkitan partikel, pembentukan *neighborhood*, dan evaluasi *fitness* awal.

Pada tahap pertama, sejumlah P partikel dibangkitkan secara acak. Untuk setiap partikel, posisi awal ditentukan dengan memilih satu sel secara acak dari domain setiap variabel warna secara independen, tanpa mempertimbangkan kendala apa pun. Dengan demikian, kandidat solusi awal dapat mengandung pelanggaran kendala baris, kolom, maupun *adjacency*. Keputusan untuk tidak menerapkan *constraint handling* pada tahap inisialisasi didasarkan pada pertimbangan bahwa mekanisme *fitness* dan tekanan selektif dari interaksi antarpartikel diharapkan mampu mengarahkan populasi menuju solusi valid secara bertahap. *Velocity* awal setiap partikel dibangkitkan secara acak dengan nilai seragam pada interval $[0, 1)$ untuk setiap dimensi.

Pada tahap kedua, seluruh partikel dipartisi ke dalam sejumlah $N_{neighborhood}$ kelompok *neighborhood* dengan ukuran yang seimbang. Partisi dilakukan dengan mengacak daftar indeks partikel menggunakan `Collections.shuffle`, kemudian membagi daftar tersebut menjadi $N_{neighborhood}$

sublista secara berurutan. Apabila P tidak habis dibagi $N_{neighborhood}$, sisa partikel didistribusikan satu per satu ke kelompok-kelompok pertama. Mekanisme *neighborhood* ini akan dibahas lebih lanjut pada Bagian 3.5.4.

Pada tahap ketiga, *fitness* setiap partikel dievaluasi menggunakan fungsi *fitness* yang didefinisikan pada Bagian 3.5.3. Nilai *fitness* awal ini sekaligus menjadi nilai **pBestFitness** masing-masing partikel, dan untuk setiap *neighborhood*, partikel dengan *fitness* terendah dicatat sebagai *neighborhood best* (**nBest**).

3.5.3 Fungsi *Fitness*

Fungsi *fitness* mengukur kualitas sebuah kandidat solusi dengan menghitung jumlah pelanggaran kendala yang terjadi pada konfigurasi penempatan menteri yang direpresentasikan oleh partikel. Nilai *fitness* yang lebih rendah menandakan kandidat solusi yang lebih baik, dan nilai *fitness* sama dengan nol menandakan solusi yang valid (seluruh kendala terpenuhi).

Pelanggaran kendala dibedakan menjadi dua komponen. Komponen pertama, *attacking violations*, menghitung jumlah pasangan menteri yang berada pada baris atau kolom yang sama. Perhitungan dilakukan dengan mengevaluasi seluruh pasangan unik (i, j) dengan $i < j$: apabila menteri ke- i dan menteri ke- j memiliki koordinat baris atau koordinat kolom yang sama, pencacah pelanggaran ditambah satu. Komponen kedua, *adjacency violations*, menghitung jumlah pasangan menteri yang bertetangga secara langsung dalam delapan arah. Untuk setiap menteri, delapan sel tetangganya diperiksa terhadap matriks **occupied**: apabila sebuah sel tetangga ditempati oleh menteri lain, pencacah pelanggaran ditambah satu. Karena setiap pasangan tetangga terhitung dua kali (sekali dari masing-masing menteri), hasil akhir dibagi dua.

Kedua komponen digabungkan melalui penjumlahan berbobot:

$$f(\mathbf{x}) = w_1 \cdot V_{attacking}(\mathbf{x}) + w_2 \cdot V_{adjacency}(\mathbf{x}) \quad (3.7)$$

di mana $\mathbf{x}$ adalah vektor posisi partikel, $V_{attacking}$ adalah jumlah pelanggaran baris/kolom, $V_{adjacency}$ adalah jumlah pelanggaran *adjacency*, dan w_1, w_2 adalah bobot yang menentukan kontribusi relatif masing-masing komponen. Pemisahan menjadi dua komponen berbobot memungkinkan eksplorasi pengaruh prioritas penalti terhadap konvergensi algoritma, yang akan dianalisis pada Bab ??.

Perlu dicatat bahwa kendala *one queen per color* tidak perlu diperiksa dalam fungsi *fitness*, karena representasi partikel sebagaimana didefinisikan pada Bagian 3.5.1 menjamin bahwa setiap sektor warna memiliki tepat satu menteri. Demikian pula, kendala diagonal tak terbatas yang ada pada N-Queens klasik tidak dievaluasi karena bukan merupakan kendala pada *Colored Queens* (lihat Bagian 3.3.4).

3.5.4 Topologi *Neighborhood*

Implementasi ini menggunakan topologi *local best* (lBest) sebagaimana dibahas pada Bagian ??, di mana partikel hanya berinteraksi dengan partikel lain dalam *neighborhood* yang sama. Pendekatan ini dipilih sebagai alternatif dari topologi *global best* (gBest) yang menghubungkan seluruh partikel ke satu *best* tunggal, dengan tujuan menjaga diversitas populasi dan mengurangi risiko konvergensi

prematur menuju *local optimum* [17].

Populasi dipartisi menjadi $N_{neighborhood}$ kelompok pada saat inisialisasi, sebagaimana dijelaskan pada Bagian 3.5.2. Setiap *neighborhood* memelihara secara independen satu *neighborhood best* (**nBest**), yaitu indeks partikel dengan nilai *fitness* terendah di dalam kelompok tersebut. Nilai **nBest** diperbarui setiap kali sebuah partikel dalam *neighborhood* mencapai *fitness* yang lebih rendah dari **nBest** saat ini.

Komposisi *neighborhood* bersifat statis sepanjang eksekusi: partisi yang ditentukan pada saat inisialisasi tidak berubah selama proses optimasi berlangsung. Konsekuensi dari desain ini adalah bahwa pertukaran informasi antarkelompok *neighborhood* tidak terjadi secara langsung. Setiap sub-populasi berevolusi secara semi-independen menuju solusi terbaiknya masing-masing. Meski pun pendekatan ini efektif dalam menjaga diversitas, ketiadaan mekanisme pertukaran informasi antarkelompok dapat memperlambat konvergensi pada kasus di mana solusi optimal memerlukan kombinasi komponen dari *neighborhood* yang berbeda. Potensi peningkatan melalui mekanisme pengacakan ulang *neighborhood* pada saat stagnasi dicatat sebagai arah pengembangan pada Bab ??.

3.5.5 Mekanisme Pembaruan Kecepatan dan Posisi

Pembaruan kecepatan dan posisi merupakan inti dari mekanisme eksplorasi PSO. Pada setiap iterasi, kedua pembaruan dieksekusi secara berurutan: seluruh kecepatan partikel diperbarui terlebih dahulu, kemudian posisi partikel diperbarui berdasarkan kecepatan baru tersebut.

Pembaruan Kecepatan

Pembaruan kecepatan mengadaptasi formula standar PSO ke dalam ruang diskrit. Untuk setiap partikel, kecepatan baru pada dimensi ke- j dihitung sebagai:

$$v_j^{(t+1)} = w \cdot v_j^{(t)} + c_1 \cdot r_1 \cdot \delta_{pBest,j} + c_2 \cdot r_2 \cdot \delta_{nBest,j} \quad (3.8)$$

di mana w adalah koefisien inersia, c_1 dan c_2 adalah koefisien akselerasi kognitif dan sosial, r_1 dan r_2 adalah bilangan acak seragam pada interval $[0, 1)$ yang dibangkitkan secara independen untuk setiap dimensi, dan $\delta_{pBest,j}$ serta $\delta_{nBest,j}$ adalah indikator perbedaan posisi yang didefinisikan sebagai:

$$\delta_{pBest,j} = \begin{cases} 1 & \text{jika } x_j^{(t)} \neq pBest_j \\ 0 & \text{jika } x_j^{(t)} = pBest_j \end{cases} \quad \delta_{nBest,j} = \begin{cases} 1 & \text{jika } x_j^{(t)} \neq nBest_j \\ 0 & \text{jika } x_j^{(t)} = nBest_j \end{cases} \quad (3.9)$$

Penggunaan indikator biner δ sebagai pengganti selisih aritmatika merupakan adaptasi kunci dari PSO kontinu ke ruang diskrit. Pada PSO kontinu, komponen kognitif dan sosial dihitung melalui selisih vektor posisi yang menghasilkan arah perpindahan dalam ruang Euclidean. Pada ruang diskrit berindeks, selisih aritmatika antara dua indeks sel tidak memiliki makna geometris: sel dengan indeks 3 dan sel dengan indeks 7 dalam domain suatu warna tidak memiliki hubungan spasial yang ditentukan oleh selisih indeksnya. Oleh sebab itu, informasi yang relevan hanyalah apakah posisi saat ini berbeda dari posisi referensi (*pBest* atau *nBest*), bukan seberapa jauh perbedaannya.

1 Setelah perhitungan, nilai kecepatan di-*clamp* pada interval $[0, 1]$ untuk mempertahankan
 2 interpretasinya sebagai probabilitas:

$$v_j^{(t+1)} = \max(0, \min(1, v_j^{(t+1)})) \quad (3.10)$$

3 Pembaruan Posisi

4 Pembaruan posisi menggunakan kecepatan yang telah diperbarui sebagai probabilitas untuk menga-
 5 dopsi posisi *neighborhood best*. Untuk setiap dimensi j , sebuah bilangan acak $r \in [0, 1)$ dibangkitkan.
 6 Apabila $r < v_j$, posisi partikel pada dimensi j diubah menjadi posisi *nBest* pada dimensi yang sama.
 7 Apabila $r \geq v_j$, posisi partikel pada dimensi j tetap tidak berubah. Secara formal:

$$x_j^{(t+1)} = \begin{cases} nBest_j & \text{jika } r < v_j^{(t+1)} \\ x_j^{(t)} & \text{jika } r \geq v_j^{(t+1)} \end{cases} \quad (3.11)$$

8 Mekanisme ini secara sengaja hanya menggunakan posisi *nBest* sebagai target perpindahan,
 9 meskipun *pBest* turut memengaruhi probabilitas perpindahan melalui formula kecepatan. Pemi-
 10 sahan peran ini merupakan keputusan desain yang didasarkan pada karakteristik ruang pencarian
 11 diskrit. Pada PSO kontinu, posisi baru merupakan hasil penjumlahan vektor yang secara alami
 12 menginterpolasi antara *pBest* dan *gBest/nBest*. Pada ruang diskrit, interpolasi semacam ini tidak
 13 terdefinisi: sebuah dimensi hanya dapat mengambil satu nilai indeks pada satu waktu, sehingga
 14 partikel harus memilih salah satu target, bukan menggabungkan keduanya. Apabila pembaruan
 15 posisi juga mempertimbangkan *pBest* sebagai target alternatif, setiap dimensi akan menghadapi
 16 dua arah perpindahan yang dapat saling bertentangan, yang dalam ruang diskrit menghasilkan
 17 pergantian indeks yang tidak koheren alih-alih konvergensi bertahap. Dengan menjadikan *nBest*
 18 sebagai satu-satunya target perpindahan, mekanisme pembaruan posisi menjadi lebih terarah,
 19 sementara pengaruh pengalaman individual partikel tetap terkodifikasi dalam besaran probabilitas
 20 melalui komponen $c_1 \cdot r_1 \cdot \delta_{pBest,j}$ pada formula kecepatan.

21 Setelah seluruh dimensi diperbarui, *fitness* partikel dievaluasi ulang. Apabila *fitness* baru
 22 lebih rendah dari *pBestFitness*, maka *pBest* dan *pBestFitness* diperbarui. Demikian pula,
 23 apabila *fitness* baru lebih rendah dari *fitness nBest* saat ini, indeks *nBest* pada *neighborhood* yang
 24 bersangkutan diperbarui.

25 Pseudocode pembaruan kecepatan dan posisi diperlihatkan pada Algoritma 8.

Algorithm 8 Pembaruan kecepatan dan posisi PSO diskrit

```

1: for setiap neighborhood  $N_i$  do
2:    $\mathbf{x}_{nBest} \leftarrow$  posisi  $\mathbf{nBest}$  dari  $N_i$ 
3:   for setiap partikel  $p$  dalam  $N_i$  do
4:                                      $\triangleright$  Pembaruan kecepatan
5:     for  $j \leftarrow 0$  hingga  $n - 1$  do
6:        $r_1, r_2 \leftarrow$  bilangan acak  $\in [0, 1)$ 
7:        $\delta_{pBest} \leftarrow \mathbf{1}[x_j \neq pBest_j]$ 
8:        $\delta_{nBest} \leftarrow \mathbf{1}[x_j \neq nBest_j]$ 
9:        $v_j \leftarrow \text{clamp}(w \cdot v_j + c_1 \cdot r_1 \cdot \delta_{pBest} + c_2 \cdot r_2 \cdot \delta_{nBest}, 0, 1)$ 
10:    end for
11:                                      $\triangleright$  Pembaruan posisi
12:    for  $j \leftarrow 0$  hingga  $n - 1$  do
13:       $r \leftarrow$  bilangan acak  $\in [0, 1)$ 
14:      if  $r < v_j$  then
15:         $x_j \leftarrow nBest_j$ 
16:      end if
17:    end for
18:    Evaluasi ulang fitness partikel  $p$ 
19:    Perbarui  $\mathbf{pBest}$  dan  $\mathbf{nBest}$  jika diperlukan
20:  end for
21: end for

```

3.5.6 Deteksi Stagnasi dan Terminasi

Algoritma PSO memerlukan kriteria terminasi yang jelas, mengingat sifat stokastiknya tidak menjamin konvergensi ke solusi optimal dalam jumlah iterasi tertentu. Dalam implementasi ini, terdapat tiga kondisi terminasi yang dievaluasi pada setiap akhir iterasi.

Kondisi pertama adalah ditemukannya solusi valid: apabila salah satu $\mathbf{nBest}$ memiliki nilai *fitness* sama dengan nol, algoritma berhenti dan melaporkan solusi tersebut. Pemeriksaan dilakukan hanya terhadap $\mathbf{nBest}$ dari setiap *neighborhood*, bukan terhadap seluruh partikel, karena partikel terbaik dalam setiap kelompok pasti merupakan $\mathbf{nBest}$ saat ini.

Kondisi kedua adalah tercapainya batas iterasi maksimum $I_{\max}$. Apabila algoritma telah berjalan selama $I_{\max}$ iterasi tanpa menemukan solusi valid, pencarian dihentikan dan partikel dengan *fitness* terendah di antara seluruh $\mathbf{nBest}$ dilaporkan sebagai solusi terbaik yang ditemukan, meskipun tidak valid.

Kondisi ketiga adalah deteksi stagnasi. Algoritma mencatat nilai *fitness* terbaik secara global (terendah di antara seluruh $\mathbf{nBest}$) pada setiap iterasi. Apabila nilai ini tidak mengalami perbaikan selama $S_{\max}$ iterasi berturut-turut, pencarian dihentikan lebih awal. Mekanisme ini menghemat waktu komputasi pada kasus di mana populasi telah terjebak pada *local optimum* dan eksplorasi lebih lanjut tidak produktif. Pencacah stagnasi direset setiap kali ditemukan *fitness* global yang lebih rendah.

Pseudocode alur utama PSO dengan ketiga kondisi terminasi diperlihatkan pada Algoritma 9.

Algorithm 9 Alur utama PSO diskrit untuk *Colored Queens*

```

1: function SOLVE
2:   Inisialisasi populasi, neighborhood, dan fitness awal
3:    $f_{globalBest} \leftarrow fitness$  terendah di antara seluruh nBest
4:   stagnasi  $\leftarrow 0$ 
5:   for iter  $\leftarrow 1$  hingga  $I_{max}$  do
6:     Perbarui kecepatan seluruh partikel
7:     Perbarui posisi dan evaluasi fitness seluruh partikel
8:     if terdapat nBest dengan fitness = 0 then
9:       return solusi valid ditemukan
10:    end if
11:     $f_{current} \leftarrow fitness$  terendah di antara seluruh nBest
12:    if  $f_{current} < f_{globalBest}$  then
13:       $f_{globalBest} \leftarrow f_{current}$ 
14:      stagnasi  $\leftarrow 0$ 
15:    else
16:      stagnasi  $\leftarrow stagnasi + 1$ 
17:    end if
18:    if stagnasi  $\geq S_{max}$  then
19:      return terminasi dini, laporkan solusi terbaik
20:    end if
21:  end for
22:  return batas iterasi tercapai, laporkan solusi terbaik
23: end function

```

1 Perlu dicatat bahwa pada kondisi terminasi kedua dan ketiga, solusi yang dilaporkan dapat
2 bersifat tidak valid (memiliki *fitness* > 0). Dalam konteks pengujian, kasus-kasus tersebut dicatat
3 sebagai kegagalan dan menjadi bagian dari metrik tingkat keberhasilan yang dianalisis pada Bab
4 ??.

5 3.5.7 Parameter PSO

6 Perilaku algoritma PSO sangat dipengaruhi oleh konfigurasi parameternya. Dalam implementasi
7 ini, terdapat sembilan parameter yang dapat dikonfigurasi melalui berkas teks yang dibaca pada
8 saat eksekusi. Tabel 3.1 merangkum seluruh parameter beserta deskripsinya.

Tabel 3.1: Parameter algoritma PSO

Parameter	Simbol	Deskripsi
Jumlah iterasi	$I_{\max}$	Batas maksimum iterasi sebelum terminasi
Jumlah partikel	P	Ukuran populasi
Jumlah <i>neighborhood</i>	N_{nh}	Jumlah kelompok dalam topologi lBest
Koefisien kognitif	c_1	Bobot pengaruh <i>personal best</i> pada kecepatan
Koefisien sosial	c_2	Bobot pengaruh <i>neighborhood best</i> pada kecepatan
Inersia	w	Bobot kecepatan sebelumnya
Bobot pelanggaran baris/kolom	w_1	Bobot komponen <i>attacking violations</i> pada fungsi <i>fitness</i>
Bobot pelanggaran <i>adjacency</i>	w_2	Bobot komponen <i>adjacency violations</i> pada fungsi <i>fitness</i>
Batas stagnasi	$S_{\max}$	Jumlah iterasi tanpa perbaikan sebelum terminasi dini

Untuk keperluan pengujian, sebuah konfigurasi *benchmark* ditetapkan sebagai *baseline* yang menjadi titik acuan seluruh eksperimen. Konfigurasi ini menggunakan nilai $P = 5000$, $N_{nh} = 100$, $c_1 = 1,0$, $c_2 = 1,0$, $w = 0,7$, $w_1 = 1,0$, $w_2 = 1,0$, $I_{\max} = 100000$, dan $S_{\max} = 500$. Pemilihan nilai-nilai *benchmark* ini didasarkan pada nilai-nilai umum yang digunakan dalam literatur PSO [17] serta penyesuaian awal terhadap karakteristik permasalahan *Colored Queens*.

Strategi pengujian menggunakan pendekatan *one-factor-at-a-time*: pada setiap eksperimen, satu parameter divariasikan sementara parameter lainnya tetap pada nilai *benchmark*. Pendekatan ini memungkinkan identifikasi pengaruh individual setiap parameter terhadap kinerja algoritma. Variasi yang diuji mencakup jumlah partikel ($P \in \{3000, 8000, 50000\}$), jumlah *neighborhood* ($N_{nh} \in \{50, 200\}$), koefisien inersia ($w \in \{0,4, 0,9\}$), koefisien kognitif ($c_1 = 1,5$), koefisien sosial ($c_2 = 1,5$), bobot pelanggaran baris/kolom ($w_1 = 2,0$), dan bobot pelanggaran *adjacency* ($w_2 = 2,0$). Selain itu, sebuah konfigurasi *optimal* disusun dengan menggabungkan nilai-nilai terbaik yang teridentifikasi dari eksperimen individual, yaitu $P = 50000$, $N_{nh} = 50$, $c_1 = 1,5$, dan $w = 0,4$, dengan parameter lainnya pada nilai *benchmark*. Rincian skenario pengujian dan analisis hasil disajikan pada Bagian 3.7 dan Bab ??.

3.6 Perancangan Perangkat Lunak

3.6.1 Arsitektur Sistem

Sistem dibangun menggunakan arsitektur modular yang memisahkan tiga tanggung jawab utama: pengelolaan data papan, logika solver, dan antarmuka pengguna. Pemisahan ini memungkinkan kedua solver beroperasi secara independen terhadap representasi visual dan sebaliknya, sehingga perubahan pada salah satu modul tidak berdampak pada modul lainnya.

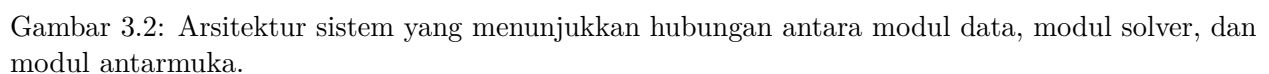
Modul pertama adalah modul data, yang terdiri atas kelas `Cell` dan `Board` dalam paket `objects` serta kelas `BoardImporter` dalam paket `util`. Kelas `Cell` merepresentasikan satu sel pada papan dengan atribut koordinat dan warna RGB. Kelas `Board` mengagregasi seluruh sel ke dalam pemetaan warna-ke-posisi menggunakan `LinkedHashMap` dan menyediakan antarmuka bagi solver

1 untuk mengakses struktur papan. Kelas **BoardImporter** bertanggung jawab membaca berkas JSON
2 dan mengonstruksi daftar objek **Cell** yang kemudian diserahkan kepada **Board**.

3 Modul kedua adalah modul solver, yang terdiri atas dua kelas dalam paket terpisah: **BacktrackingSolverAC3**
4 dalam paket **solver.backtracking** dan **PSOSolver** dalam paket **solver.PSO**. Kedua kelas meneri-
5 ma objek **Board** sebagai masukan dan menghasilkan solusi berupa koordinat penempatan menteri
6 untuk setiap warna. Kedua solver tidak memiliki ketergantungan terhadap modul antarmuka,
7 sehingga dapat dieksekusi secara mandiri melalui *command line* untuk keperluan pengujian.

8 Modul ketiga adalah modul antarmuka dalam paket **GUI**, yang terdiri atas empat kelas: **Main**
9 sebagai titik masuk aplikasi, **Homepage** sebagai halaman utama, **LevelSelectorWindow** sebagai
10 halaman pemilihan level, dan **GameWindow** sebagai halaman permainan utama. Kelas **UITheme**
11 menyediakan konstanta visual (palet warna, font, komponen tombol) yang digunakan secara
12 konsisten di seluruh antarmuka.

13 Diagram hubungan antarmodul diperlihatkan pada Gambar [3.2](#).



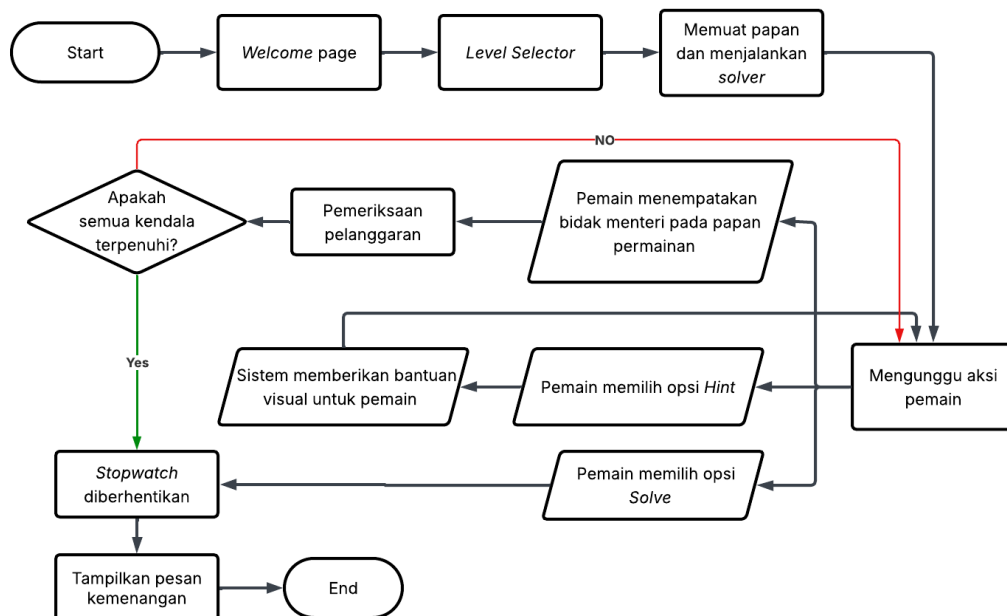
Struktur program diorganisasi ke dalam empat paket Java yang mencerminkan pembagian modul pada arsitektur sistem. Tabel 3.2 merangkum seluruh kelas beserta tanggung jawab utamanya.

Tabel 3.2: Struktur kelas dalam sistem

Paket	Kelas	Tanggung Jawab
app	Main	Titik masuk aplikasi; menginisialisasi Homepage pada <i>Event Dispatch Thread</i>
objects	Cell	Representasi satu sel papan (koordinat + warna RGB)
	Board	Representasi papan keseluruhan; pemetaan warna ke daftar posisi sel
util	BoardImporter	Pembacaan dan penguraian berkas JSON papan
solver	BacktrackingSolverAC3	Solver berbasis <i>backtracking</i> dengan <i>forward checking</i> dan AC-3
	PSOSolver	Solver berbasis PSO diskrit (termasuk kelas <i>Particle</i>)
GUI	Homepage	Halaman utama dengan tombol navigasi
	LevelSelectorWindow	Halaman pemilihan ukuran papan dan level
	GameWindow	Halaman permainan utama; menampilkan papan, mengintegrasikan solver, dan mengelola interaksi pengguna
	UITheme	Konstanta visual (palet warna, font, komponen tombol)

3.6.3 Antarmuka Pengguna

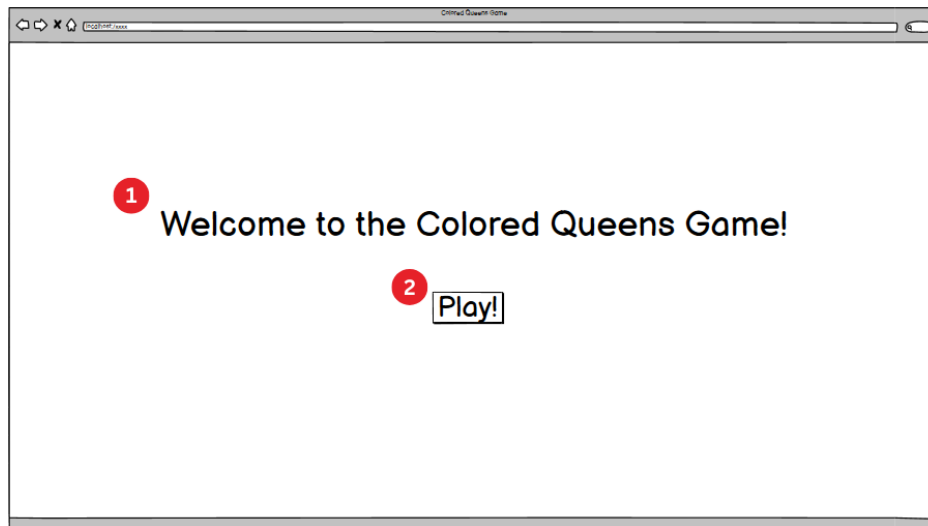
Antarmuka pengguna terdiri atas tiga layar utama yang dinavigasi secara berurutan. Alur interaksi pengguna dengan aplikasi diilustrasikan pada Gambar 3.3.

Gambar 3.3: Alur interaksi pengguna pada aplikasi *Colored Queens*.

Berikut adalah penjelasan setiap proses pada Gambar 3.3:

1. **Welcome Page.** Layar pertama (Homepage) menampilkan judul permainan, ikon menteri, dan tombol *Start Game* yang mengarahkan pengguna ke halaman pemilihan level. Halaman ini berfungsi sebagai titik masuk visual dan tidak memuat logika permainan.
2. **Level Selector.** Layar kedua (LevelSelectorWindow) menyajikan pilihan ukuran papan yang dikelompokkan berdasarkan tingkat kesulitan: *Easy* (7×7), *Medium* (8×8 dan 9×9), serta *Hard* (10×10 , 11×11 , dan 12×12). Selain itu, terdapat kategori *Challenge* yang menyediakan papan berukuran 20×20 dan 30×30 dengan masing-masing satu level. Pemilihan ukuran papan langsung membuka halaman permainan pada level pertama dari ukuran tersebut.
3. **Memuat papan dan menjalankan solver.** Setelah pengguna memilih level, kelas `GameWindow` membaca berkas JSON melalui `BoardImporter`, mengonstruksi objek `Board`, dan menjalankan solver *backtracking* secara asinkron menggunakan `SwingWorker`. Solver berjalan di latar belakang agar antarmuka tetap responsif selama proses pencarian berlangsung. Untuk papan berukuran 20×20 dan 30×30 , solusi disimpan secara *hardcoded* karena waktu komputasi solver pada ukuran tersebut terlalu lama untuk dieksekusi secara *on-the-fly*. Solusi yang dihasilkan disimpan dan digunakan oleh fitur *hint* dan *auto-solve*.
4. **Menunggu aksi pemain.** Sistem memasuki kondisi menunggu, di mana papan ditampilkan sebagai grid berwarna di tengah jendela disertai panel kontrol di bagian bawah. Dari kondisi ini, pengguna dapat melakukan salah satu dari tiga aksi berikut.
5. **Pemain menempatkan bidak menteri pada papan permainan.** Pengguna menempatkan atau menghapus menteri dengan mengklik sel pada papan. Pencacah waktu mulai berjalan saat pengguna menempatkan menteri pertama.
6. **Pemeriksaan pelanggaran.** Setiap kali pengguna menempatkan atau menghapus menteri, sistem memeriksa seluruh pasangan menteri terhadap kendala baris, kolom, *adjacency*, dan kesamaan warna. Menteri yang terlibat dalam pelanggaran ditandai dengan ikon berwarna merah.
7. **Apakah semua kendala terpenuhi?** Sistem memeriksa apakah jumlah menteri yang ditempatkan sama dengan ukuran papan dan seluruh posisi sesuai dengan solusi. Apabila tidak, sistem kembali ke kondisi menunggu aksi pemain.
8. **Pemain memilih opsi Hint.** Sistem menampilkan petunjuk visual berupa sorotan hijau pada posisi yang benar dan sorotan oranye pada posisi yang salah untuk satu sektor warna. Setelah petunjuk ditampilkan, sistem kembali ke kondisi menunggu aksi pemain.
9. **Pemain memilih opsi Solve.** Solusi lengkap yang telah dihitung di latar belakang diterapkan secara otomatis pada papan.
10. **Stopwatch dihentikan dan pesan kemenangan ditampilkan.** Pencacah waktu dihentikan dan dialog kemenangan ditampilkan kepada pengguna.

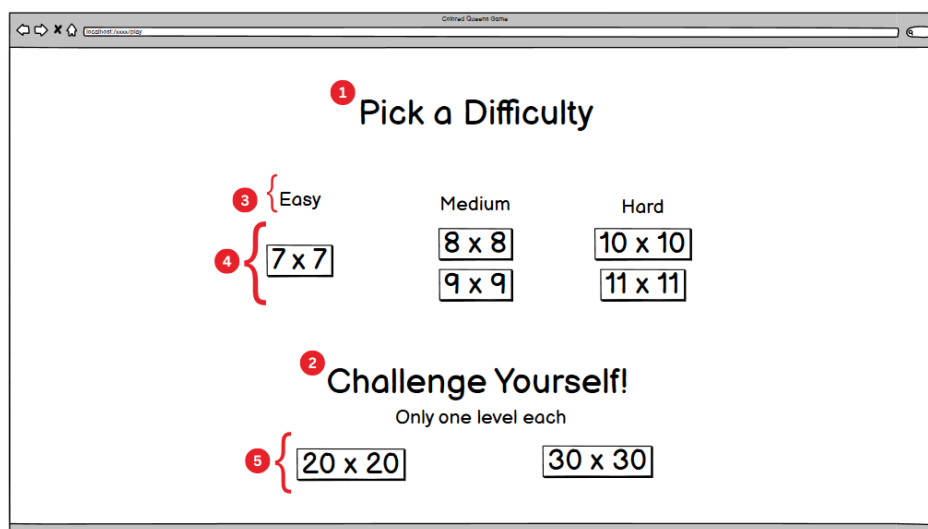
Rancangan antarmuka untuk setiap layar beserta elemen-elemen interaktifnya diperlihatkan pada Gambar 3.4 hingga Gambar 3.14.



Gambar 3.4: Rancangan halaman utama (*Homepage*).

14 Penjelasan elemen pada Gambar 3.4:

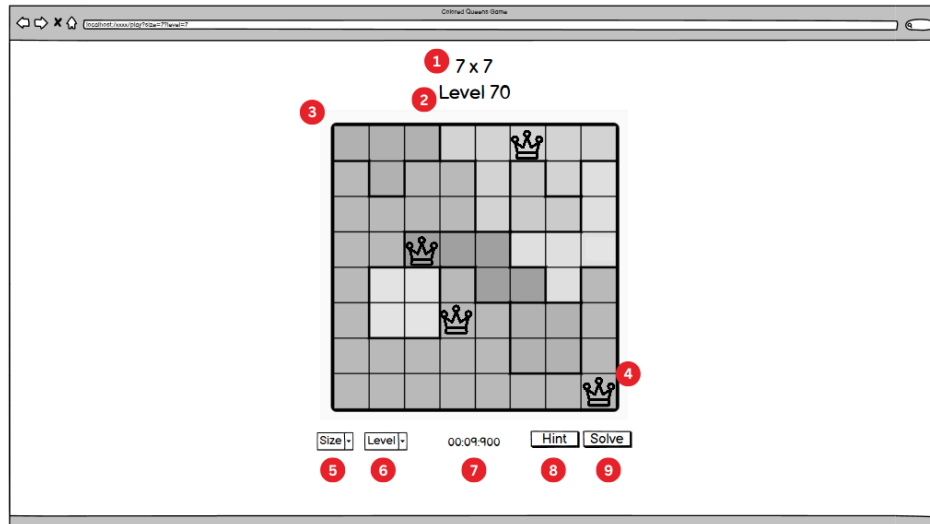
- 15 1. Judul aplikasi yang menampilkan nama permainan.
16 2. Tombol *Play* yang mengarahkan pengguna ke halaman pemilihan level.



Gambar 3.5: Rancangan halaman pemilihan level (*Level Selector*).

1 Penjelasan elemen pada Gambar 3.5:

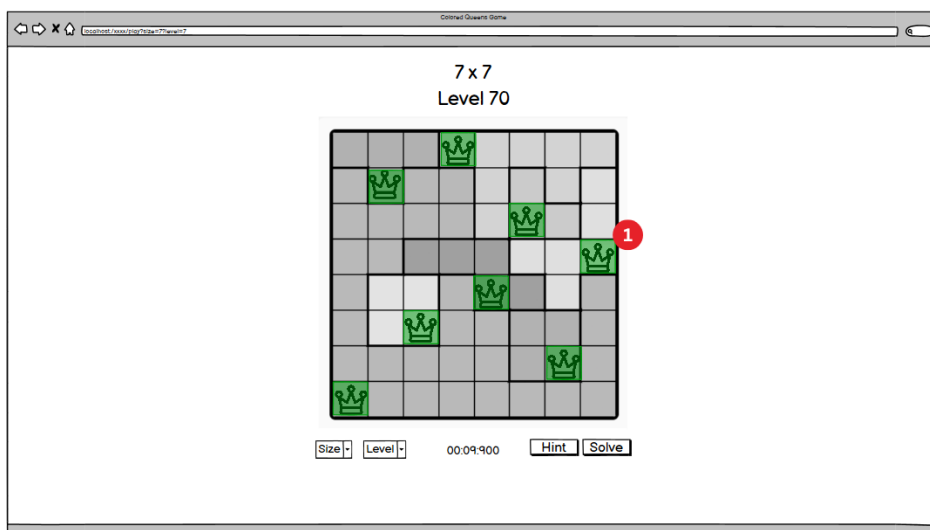
- 2 1. Judul bagian pemilihan tingkat kesulitan.
3 2. Judul bagian tantangan (*Challenge*) untuk papan berukuran besar.
4 3. Label kategori kesulitan (*Easy, Medium, Hard*) yang mengelompokkan ukuran papan.
5 4. Tombol ukuran papan yang dapat dipilih oleh pengguna untuk memulai permainan.
6 5. Tombol ukuran papan pada kategori *Challenge* (20×20 dan 30×30).



Gambar 3.6: Rancangan halaman permainan (*Game Window*).

Penjelasan elemen pada Gambar 3.6:

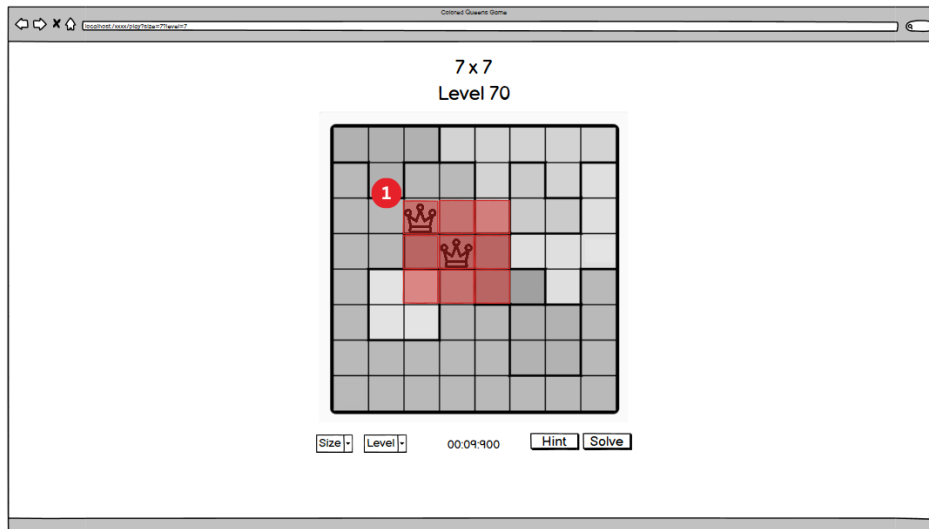
1. Label ukuran papan yang sedang dimainkan.
2. Label nomor level yang sedang dimainkan.
3. Papan permainan berupa grid berwarna tempat pengguna menempatkan bidak menteri.
4. Bidak menteri yang telah ditempatkan pada papan. Ikon berwarna hitam digunakan pada sel terang dan ikon berwarna putih pada sel gelap, dengan pemilihan berdasarkan luminansi warna sel.
5. *Dropdown* ukuran papan untuk berpindah ke ukuran lain.
6. *Dropdown* level untuk berpindah ke level lain dalam ukuran yang sama.
7. Pencacah waktu (*stopwatch*) yang mulai berjalan saat pengguna menempatkan menteri pertama.
8. Tombol *Hint* yang menampilkan petunjuk visual.
9. Tombol *Solve* yang menerapkan solusi lengkap secara otomatis.



Gambar 3.7: Tampilan papan setelah solver menerapkan solusi lengkap.

Penjelasan elemen pada Gambar 3.7:

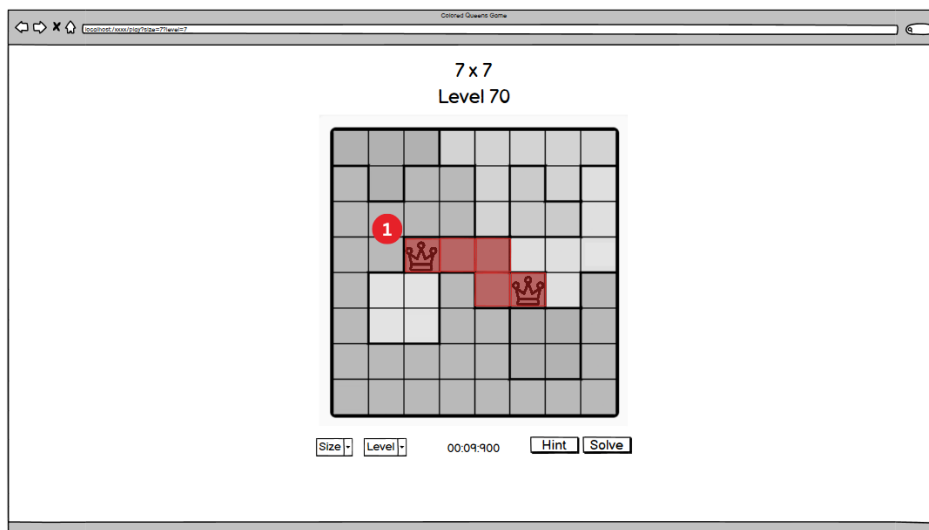
1. Seluruh bidak menteri ditempatkan pada posisi yang benar oleh solver. Seluruh menteri ditampilkan dengan warna hijau untuk menandakan solusi yang valid.



Gambar 3.8: Tampilan peringatan pelanggaran kendala *adjacency*.

Penjelasan elemen pada Gambar 3.8:

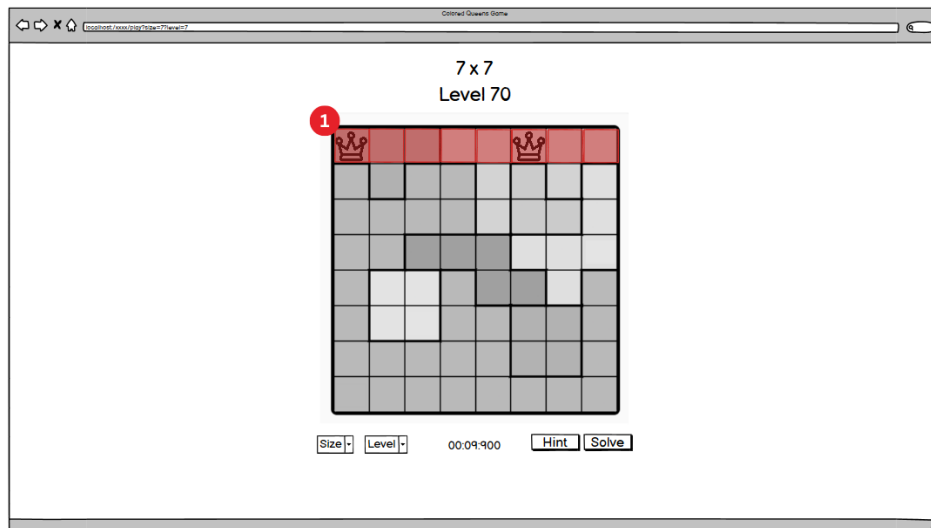
1. Dua bidak menteri yang bertetangga secara diagonal ditandai berwarna merah, menunjukkan pelanggaran kendala *adjacency*. Sel-sel di sekitar kedua menteri yang berkonflik juga disorot untuk memperjelas area pelanggaran.



Gambar 3.9: Tampilan peringatan pelanggaran kendala kesamaan warna.

Penjelasan elemen pada Gambar 3.9:

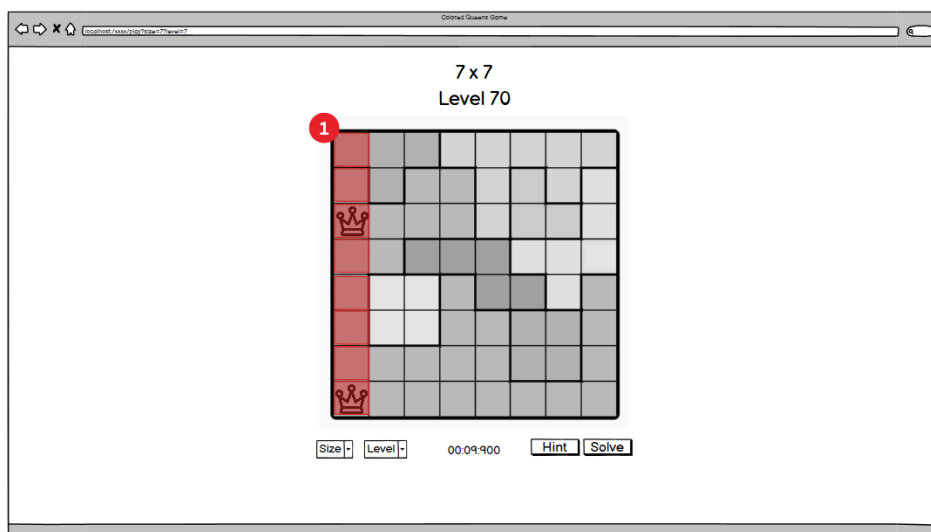
1. Dua bidak menteri yang ditempatkan pada sektor warna yang sama ditandai berwarna merah, menunjukkan pelanggaran kendala *one queen per color*.



Gambar 3.10: Tampilan peringatan pelanggaran kendala baris.

4 Penjelasan elemen pada Gambar 3.10:

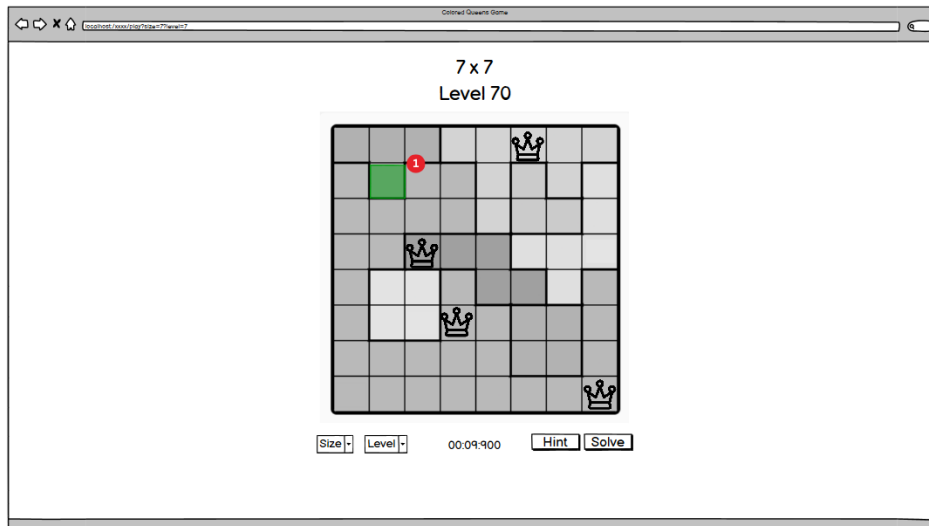
- 5 1. Dua bidak menteri yang berada pada baris yang sama ditandai berwarna merah. Seluruh sel
- 6 pada baris tersebut disorot untuk memperjelas area pelanggaran.



Gambar 3.11: Tampilan peringatan pelanggaran kendala kolom.

1 Penjelasan elemen pada Gambar 3.11:

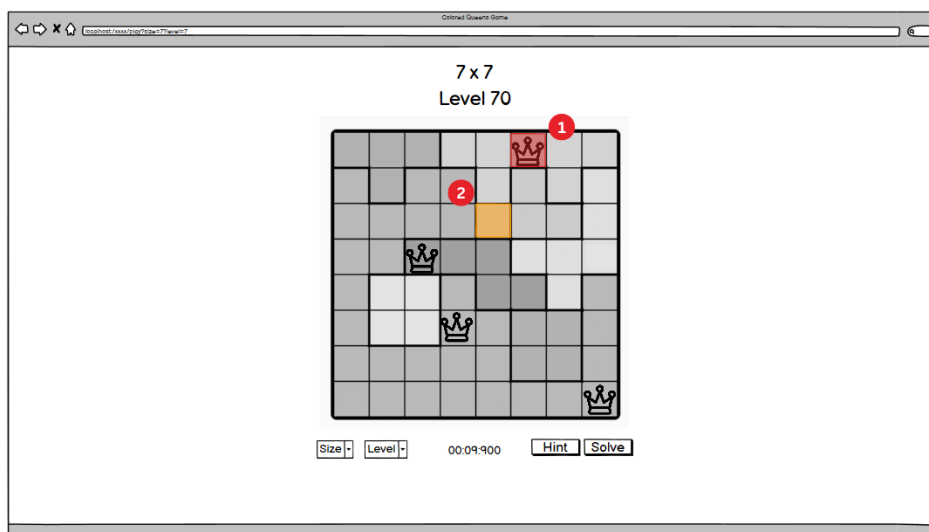
- 2 1. Dua bidak menteri yang berada pada kolom yang sama ditandai berwarna merah. Seluruh sel
- 3 pada kolom tersebut disorot untuk memperjelas area pelanggaran.



Gambar 3.12: Tampilan fitur *hint*: sorotan posisi yang benar.

Penjelasan elemen pada Gambar 3.12:

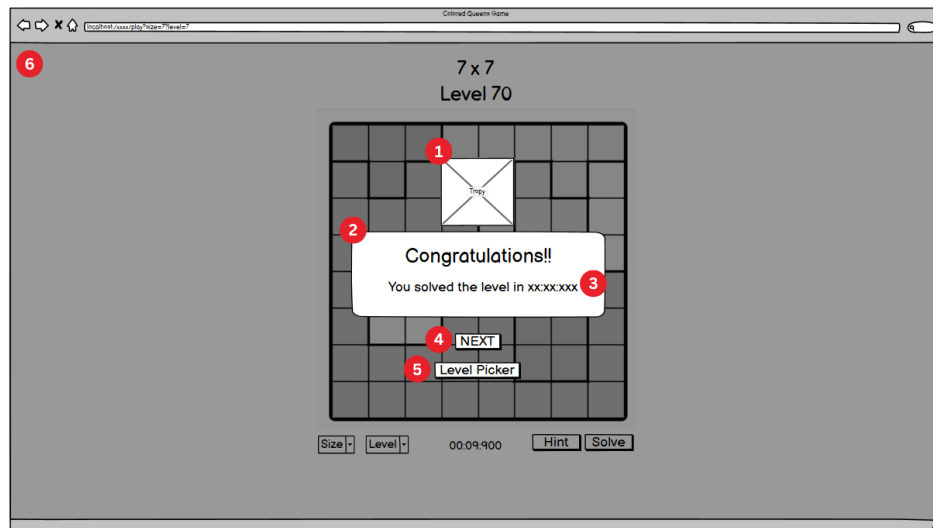
1. Sorotan hijau pada sel yang merupakan posisi benar untuk sektor warna yang belum terisi, menunjukkan di mana menteri seharusnya ditempatkan.



Gambar 3.13: Tampilan fitur *hint*: sorotan posisi salah dan posisi benar.

Penjelasan elemen pada Gambar 3.13:

1. Menteri yang ditempatkan pada posisi salah ditandai dengan sorotan merah, menunjukkan bahwa penempatan perlu dipindahkan.
2. Sorotan oranye pada sel yang merupakan posisi benar untuk sektor warna tersebut, menunjukkan ke mana menteri seharusnya dipindahkan.



Gambar 3.14: Rancangan tampilan kemenangan.

Penjelasan elemen pada Gambar 3.14:

1. Ikon trofi sebagai elemen visual perayaan kemenangan.
2. Pesan ucapan selamat kepada pemain.
3. Waktu penyelesaian yang dicatat oleh *stopwatch* selama permainan berlangsung.
4. Tombol *Next* untuk melanjutkan ke level berikutnya.
5. Tombol *Level Picker* untuk kembali ke halaman pemilihan level.
6. Latar belakang halaman permainan yang diredupkan (*dimmed*) untuk memberikan fokus pada dialog kemenangan.

3.6.4 Teknologi yang Digunakan

Tabel 3.3 merangkum teknologi dan pustaka yang digunakan dalam pengembangan sistem.

Tabel 3.3: Teknologi yang digunakan

Teknologi	Penggunaan
Java (JDK 23)	Bahasa pemrograman utama untuk seluruh modul
Java Swing / JFrame	<i>Framework</i> antarmuka grafis untuk rendering papan, navigasi, dan interaksi pengguna
<code>org.json</code>	Pustaka untuk penguraian berkas JSON papan
JSON	Format penyimpanan data papan (<code>NxN_levelM.json</code>)

Pemilihan Java Swing sebagai *itframework* antarmuka didasarkan pada kemudahan implementasi dan ketersediaannya sebagai bagian dari *Java Standard Edition* tanpa memerlukan dependensi eksternal tambahan. Meskipun Swing merupakan teknologi yang lebih lama dibandingkan alternatif seperti JavaFX, kemampuannya untuk merender grid berwarna dan menangani interaksi klik pengguna cukup memadai untuk kebutuhan visualisasi papan *Colored Queens*.

3.7 Skenario dan Metode Pengujian

3.7.1 Dataset Puzzle

Pengujian dilakukan pada delapan ukuran papan *Colored Queens*. Untuk ukuran standar (7×7 hingga 12×12), masing-masing memiliki 250 level puzzle yang diuji secara keseluruhan. Untuk ukuran besar (20×20 dan 30×30), masing-masing hanya memiliki satu level. Tabel 3.4 merangkum seluruh dataset yang digunakan.

Tabel 3.4: Dataset puzzle yang digunakan dalam pengujian

Ukuran Papan	Kategori	Jumlah Level
7×7	<i>Easy</i>	250
8×8	<i>Medium</i>	250
9×9	<i>Medium</i>	250
10×10	<i>Hard</i>	250
11×11	<i>Hard</i>	250
12×12	<i>Hard</i>	250
20×20	<i>Challenge</i>	1
30×30	<i>Challenge</i>	1

Total dataset untuk ukuran standar berjumlah 1.500 puzzle (6×250), ditambah 2 puzzle untuk ukuran besar, sehingga keseluruhan dataset mencakup 1.502 puzzle. Seluruh data papan bersumber dari permainan *LinkedIn Queens* dan disimpan dalam format JSON. Variasi ukuran papan dipilih untuk menguji skalabilitas kedua algoritma: papan berukuran kecil (7×7 hingga 9×9) memiliki ruang pencarian yang relatif terbatas, sedangkan papan berukuran besar (20×20 dan 30×30) menghadirkan tantangan kombinatorik yang signifikan.

3.7.2 Parameter Pengujian

Kinerja kedua algoritma dievaluasi berdasarkan empat metrik utama:

- Waktu eksekusi:** durasi yang diperlukan solver untuk menyelesaikan satu puzzle, diukur dalam milidetik menggunakan `System.currentTimeMillis()`.
- Tingkat keberhasilan:** persentase eksekusi yang menghasilkan solusi valid (seluruh kendala terpenuhi). Untuk solver *backtracking*, metrik ini secara teoretis selalu bernilai 100% apabila solusi ada, karena sifat pencarian yang lengkap (*complete*). Untuk PSO, metrik ini bergantung pada konfigurasi parameter dan ukuran papan.
- Jumlah langkah/iterasi:** untuk *backtracking*, jumlah penugasan yang berhasil dilanjutkan ke tingkat rekursi berikutnya (*steps*) dan jumlah pembatalan penugasan (*backtracks*). Untuk PSO, jumlah iterasi yang dieksekusi sebelum terminasi.
- Nilai *fitness* akhir:** khusus untuk PSO, nilai fungsi *fitness* dari solusi terbaik yang ditemukan. Nilai nol menandakan solusi valid; nilai positif menandakan jumlah pelanggaran yang tersisa, termasuk rincian *adjacency violations* dan *attacking violations*.

3.7.3 Skenario Pengujian

Pengujian dirancang dalam tiga kelompok skenario yang masing-masing menguji aspek yang berbeda.

Skenario 1: Pengujian *Backtracking*. Solver *backtracking* dengan *forward checking* dan AC-3 dijalankan pada seluruh 1.502 puzzle untuk mengukur waktu eksekusi, jumlah langkah, dan jumlah *backtracks*. Karena solver ini bersifat deterministik, setiap puzzle hanya perlu dijalankan satu kali.

Skenario 2: Pengujian variasi parameter PSO. Solver PSO dijalankan dengan 13 konfigurasi parameter sebagaimana dirangkum pada Tabel 3.5. Setiap konfigurasi dijalankan pada seluruh dataset. Untuk ukuran standar, 250 level per ukuran dieksekusi dalam 5 *batch* yang masing-masing berisi 50 level. Eksekusi *batch* dilakukan secara paralel menggunakan *ExecutorService* dengan 6 *thread* untuk mempercepat proses pengujian. Untuk ukuran besar (20×20 dan 30×30), hanya PSO yang dijalankan (tanpa *backtracking*) karena waktu komputasi solver deterministik pada ukuran tersebut tidak praktis. Hasil setiap konfigurasi diagregasi per *batch* dan per ukuran papan untuk menghasilkan metrik rata-rata waktu eksekusi dan tingkat keberhasilan.

Tabel 3.5: Konfigurasi pengujian PSO

Nama Konfigurasi	Parameter yang Diubah	Nilai
Benchmark	(baseline)	$P=5000$, $N_{nh}=100$, $c_1=1,0$, $c_2=1,0$, $w=0,7$, $w_1=1,0$, $w_2=1,0$
LParticles	Jumlah partikel	$P = 3000$
MParticles	Jumlah partikel	$P = 8000$
MMParticles	Jumlah partikel	$P = 50000$
LNeighborhoods	Jumlah <i>neighborhood</i>	$N_{nh} = 50$
MNeighborhoods	Jumlah <i>neighborhood</i>	$N_{nh} = 200$
LIInertia	Koefisien inersia	$w = 0,4$
MInertia	Koefisien inersia	$w = 0,9$
c1	Koefisien kognitif	$c_1 = 1,5$
c2	Koefisien sosial	$c_2 = 1,5$
w1	Bobot <i>attacking</i>	$w_1 = 2,0$
w2	Bobot <i>adjacency</i>	$w_2 = 2,0$
Optimal	Kombinasi terbaik	$P=50000$, $N_{nh}=50$, $c_1=1,5$, $w=0,4$

Skenario 3: Perbandingan *Backtracking* vs PSO. Hasil solver *backtracking* dan konfigurasi PSO terbaik (Optimal) dibandingkan secara langsung pada seluruh ukuran papan standar, dengan fokus pada waktu eksekusi dan tingkat keberhasilan. Perbandingan ini bertujuan untuk mengidentifikasi kelebihan dan keterbatasan masing-masing pendekatan pada skala permasalahan yang berbeda.

3.7.4 Lingkungan Pengujian

Seluruh pengujian dilakukan pada satu mesin dengan spesifikasi yang ditunjukkan pada Tabel 3.6. Penggunaan satu lingkungan pengujian yang konsisten memastikan bahwa perbedaan waktu eksekusi antarkonfigurasi tidak dipengaruhi oleh variasi perangkat keras.

Tabel 3.6: Spesifikasi lingkungan pengujian

Komponen	Spesifikasi
Sistem Operasi	Windows 11 Home
Prosesor	Intel i5-12400F
Memori (RAM)	32GB DDR4 @ 3200 MHz
Java Version	23
IDE	Visual Studio Code

DAFTAR REFERENSI

- [1] Bell, J. dan Stevens, B. (2009) A survey of known results and research areas for n-queens. *Discrete mathematics*, **309**, 1–31. <https://www.sciencedirect.com/science/article/pii/S0012365X07010394#b20> Diakses: 2026-05-10.
- [2] Glock, S., Munhá Correia, D., dan Sudakov, B. (2022) The n-queens completion problem. *Research in the Mathematical Sciences*, **9**, 41. <https://link.springer.com/article/10.1007/s40687-022-00335-1> Diakses: 2026-05-10.
- [3] Russell, S. J. dan Norvig, P. (2020) *Artificial Intelligence: A Modern Approach*, 4 edition. Pearson Education, Inc., New Jersey. Chapter 6. [http://repo.darmajaya.ac.id/5272/1/Artificial%20Intelligence-A%20Modern%20Approach%20\(3rd%20Edition\)%20\(%20PDFDrive%20\).pdf](http://repo.darmajaya.ac.id/5272/1/Artificial%20Intelligence-A%20Modern%20Approach%20(3rd%20Edition)%20(%20PDFDrive%20).pdf) Diakses: 2026-05-10.
- [4] Hoffman, E. J., Loessi, J., dan Moore, R. C. (1969) Constructions for the solution of the m queens problem. *Mathematics Magazine*, **42**, 66–72. <https://exp-blog.com/algorithm/poj/poj3239-solution-to-the-n-queens-puzzle/doc/Constructions%20for%20the%20Solution%20of%20the%20m%20Queens%20Problem.pdf> Diakses: 2026-05-14.
- [5] Gent, I. P., Jefferson, C., dan Nightingale, P. (2017) Complexity of n-queens completion. *Journal of Artificial Intelligence Research*, **59**, 815–848. <https://www.jair.org/index.php/jair/article/view/11079/26262> Diakses: 2026-05-14.
- [6] LinkedIn (2024) Play queens game on linkedin. <https://www.linkedin.com/help/linkedin/answer/a6269510>. Diakses: 2026-05-14.
- [7] Snyder, T. (2013) Star battle rules and info. <https://www.gmpuzzles.com/blog/star-battle-rules-and-info/>. Diakses: 14 Mei 2026.
- [8] Ali, A. M. dan Mencia, E. (2026) A qubo formulation for the generalized linkedin queens and takuzu/tango game.
- [9] Montanari, U. (1974) Networks of constraints: Fundamental properties and applications to picture processing. *Information sciences*, **7**, 95–132. <http://cse.unl.edu/~choueiry/Documents/Montanari-74.pdf> Diakses 14 Mei 2026.
- [10] Kumar, V. (1998) Algorithms for constraint satisfaction problems: A survey. *A.I. Mag*, **13**.
- [11] Van Beek, P. (2006) Backtracking search algorithms. *Handbook of Constraint Programming*, pp. 85–134. Elsevier. <https://cs.uwaterloo.ca/~vanbeek/Publications/survey06.pdf> Diakses: 17 Mei 2026.
- [12] Mackworth, A. K. (1977) Consistency in networks of relations. *Artificial intelligence*, **8**, 99–118. <https://www.cs.ubc.ca/~mack/Publications/AI77.pdf> Diakses: 17 Mei 2026.
- [13] Mohr, R. dan Henderson, T. C. (1986) Arc and path consistency revisited. *Artificial intelligence*, **28**, 225–233. <https://sci-hub.mk/10.1016/0004-3702%2886%2990083-4> Diakses: 17 Mei 2026.

-
- [14] Bessiere, C. (2006) Constraint propagation. *Handbook of Constraint Programming*, pp. 29–83. Elsevier. https://www.dcs.gla.ac.uk/~pat/cpM/papers/CP_Handbook-20060315-final.pdf Diakses: 17 Mei 2026.
- [15] Blum, C. dan Roli, A. (2003) Metaheuristics in combinatorial optimization: Overview and conceptual comparison. *ACM computing surveys (CSUR)*, **35**, 268–308. <https://dl.acm.org/doi/epdf/10.1145/937503.937505> Diakses: 17 Mei 2026.
- [16] Hoos, H. H. dan Tsang, E. (2006) Local search methods. *Handbook of Constraint Programming*, pp. 245–268. Elsevier. https://www.dcs.gla.ac.uk/~pat/cpM/papers/CP_Handbook-20060315-final.pdf Diakses: 17 Mei 2026.
- [17] Kennedy, J. dan Eberhart, R. (1995) Particle swarm optimization. *Proceedings of ICNN'95-international conference on neural networks*, pp. 1942–1948. iee. https://www.cs.tufts.edu/comp/150GA/homeworks/hw3/_reading6%201995%20particle%20swarming.pdf Diakses: 18 Mei 2026.
- [18] Hu, X., Eberhart, R. C., dan Shi, Y. (2003) Swarm intelligence for permutation optimization: a case study of n-queens problem. *Proceedings of the 2003 IEEE Swarm Intelligence Symposium. SIS'03 (Cat. No. 03EX706)*, pp. 243–246. IEEE. <http://www.swarmintelligence.org/papers/SIS2003Permutation.pdf> Diakses: 18 Mei 2026.
- [19] Kennedy, J. dan Eberhart, R. C. (1997) A discrete binary version of the particle swarm algorithm. *1997 IEEE International conference on systems, man, and cybernetics. Computational cybernetics and simulation*, pp. 4104–4108. iee. https://www.ahmetcevahircinar.com.tr/wp-content/uploads/2017/02/A_discrete_binary_version_of_the_particle_swarm_algorithm.pdf Diakses: 18 Mei 2026.

LAMPIRAN A

KODE PROGRAM

Kode A.1: MyCode.c

```
1 // This does not make algorithmic sense,
2 // but it shows off significant programming characters.
3
4 #include<stdio.h>
5
6 void myFunction( int input, float* output ) {
7     switch ( array[i] ) {
8         case 1: // This is silly code
9             if ( a >= 0 || b <= 3 && c != x )
10                 *output += 0.005 + 20050;
11             char = 'g';
12             b = 2^n + ~right_size - leftSize * MAX_SIZE;
13             c = (--aaa + &daa) / (bbb++ - ccc % 2 );
14             strcpy(a,"hello_$@?");
15         }
16         count = ~mask | 0x00FF00AA;
17     }
18 }
19
20 // Fonts for Displaying Program Code in LATEX
21 // Adrian P. Robson, nepsweb.co.uk
22 // 8 October 2012
23 // http://nepsweb.co.uk/docs/progfonts.pdf
```

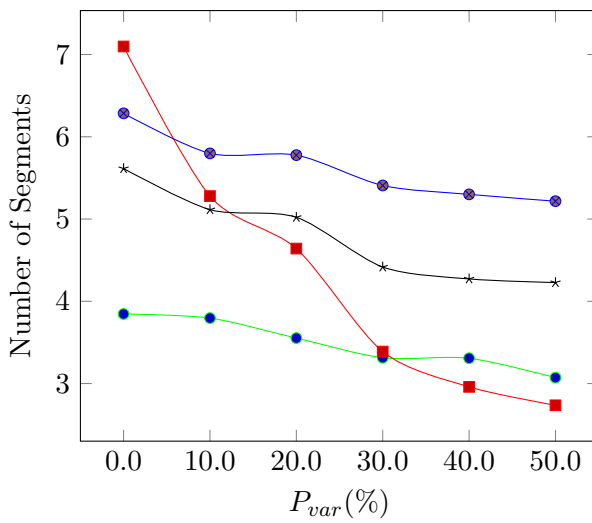
Kode A.2: MyCode.java

```
1 import java.util.ArrayList;
2 import java.util.Collections;
3 import java.util.HashSet;
4
5 //class for set of vertices close to furthest edge
6 public class MyFurSet {
7     protected int id; //id of the set
8     protected MyEdge FurthestEdge; //the furthest edge
9     protected HashSet<MyVertex> set; //set of vertices close to furthest edge
10    protected ArrayList<ArrayList<Integer>> ordered; //list of all vertices in the set for each trajectory
11    protected ArrayList<Integer> closeID; //store the ID of all vertices
12    protected ArrayList<Double> closeDist; //store the distance of all vertices
13    protected int totaltrj; //total trajectories in the set
14
15    /*
16     * Constructor
17     * @param id : id of the set
18     * @param totaltrj : total number of trajectories in the set
19     * @param FurthestEdge : the furthest edge
20     */
21    public MyFurSet(int id,int totaltrj,MyEdge FurthestEdge) {
22        this.id = id;
23        this.totaltrj = totaltrj;
24        this.FurthestEdge = FurthestEdge;
25        set = new HashSet<MyVertex>();
26        ordered = new ArrayList<ArrayList<Integer>>();
27        for (int i=0;i<totaltrj;i++) ordered.add(new ArrayList<Integer>());
28        closeID = new ArrayList<Integer>(totaltrj);
29        closeDist = new ArrayList<Double>(totaltrj);
30        for (int i = 0;i <totaltrj;i++) {
31            closeID.add(-1);
32            closeDist.add(Double.MAX_VALUE);
33        }
34    }
35
36 }
```

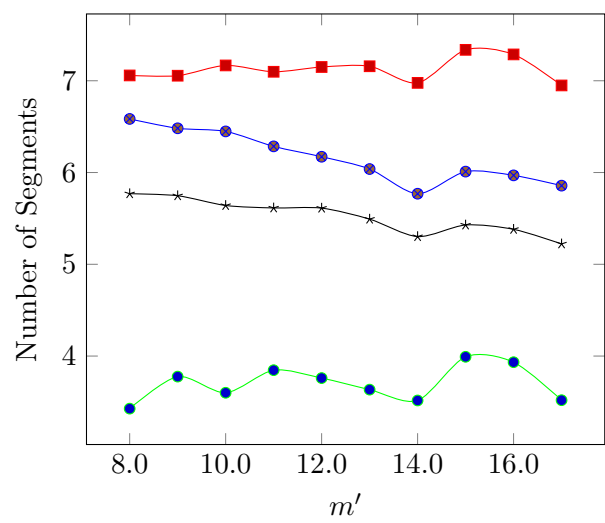

LAMPIRAN B

HASIL EKSPERIMEN

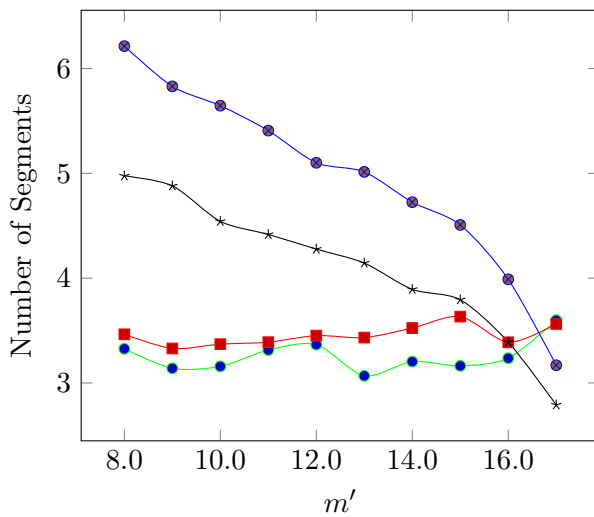
Hasil eksperimen berikut dibuat dengan menggunakan TIKZPICTURE (bukan hasil excel yg diubah ke file bitmap). Sangat berguna jika ingin menampilkan tabel (yang kuantitasnya sangat banyak) yang datanya dihasilkan dari program komputer.



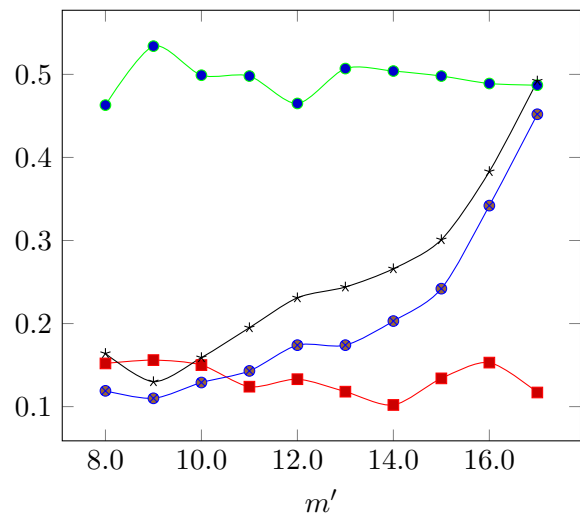
Gambar B.1: Hasil 1



Gambar B.2: Hasil 2



Gambar B.3: Hasil 3



Gambar B.4: Hasil 4